Soli Testing App

A Project Report
Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By

Lokhande Saniya Vilas
259663

Under the guidance of
Dr. Jyotika Chheda
Co-ordinator

SCHOOL OF COMPUTING & TECHNOLOGY
PARLE TILAK VIDYALAYA ASSOCIATION'S
MULUND COLLEGE OF COMMERCE (AUTONOMOUS)
(Affiliated to University of Mumbai)
MULUND – 400 080
MAHARASHTRA
2025-26

ACKNOWLEDGEMENT

I like to extend my gratitude to the I/C Principal Dr. Shivaji Pawar and all staff of Mulund College of Commerce (Autonomous) for providing us moral support, the required infrastructure and resource to enable the completion and enrichment of my project.

I also take this opportunity to thank the School of Computing & Technology Coordinator, Dr. Reena Nagda, Programme Coordinator, Dr. Jyotika Chheda and all the faculty of School of Computing & Technology for giving us the most needed guidance and continuous encouragement throughout the duration of the Programme.

I wish to extend my deepest gratitude and special thanks to my project guide Dr. Jyotika Chheda, for the guidance and the inspiration to complete this project on time. I perceive the skills and knowledge which I have gained throughout my project work as a very valuable component in my future career development

I am also extremely grateful to the University of Mumbai for having prescribed this project work to me as a part of the academic requirement.

Finally, I thank all my friends and family members who have directly or indirectly helped me in completing my project.

Project Declaration

1.	Student Name	Saniya Vilas Lokhande
2.	Roll No	259663
3.	Class	T.Y. B.Sc. Information Technology
4.	Department	Information Technology
5.	APAAR ID	561394348400

I hereby declare that the project entitled, "**Soil Testing App**", is my own work, based on my personal study and/or research and that I have acknowledged all material and sources used in its preparation, whether they be books, articles, reports, lecture notes, and any other kind of document, electronic or personal communication.

I certify that I have not copied, in part or whole, nor otherwise plagiarised the work of any other person.

The project is done in partial fulfilment of the requirements for the award of degree of **BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)** to be submitted as Semester VI project as part of our curriculum.

Student's Signature

ABSTRACT

A Soil Testing App is a modern digital solution designed to make soil analysis faster, easier, and more accessible for farmers. It removes the dependency on traditional laboratory testing by offering instant soil evaluation directly through a smartphone. Soil quality plays a crucial role in agricultural productivity, yet conventional testing methods are often expensive, slow, and unavailable in many rural areas. Beyond its core functionality, the Soil Testing Web Application also focuses on enhancing user awareness by simplifying complex agricultural data into easy-to-understand insights. Users not only receive recommendations but also learn *why* certain nutrients are important, how soil pH affects plant growth, and what steps can be taken to restore soil balance. The application's intuitive interface ensures that even individuals with minimal technical or agricultural knowledge can effortlessly navigate the system and interpret their soil health results.

To support long-term agricultural planning, the system includes seasonal guidance and crop rotation suggestions, allowing users to select the best crops for upcoming cycles based on historical data and weather patterns. This helps prevent soil nutrient depletion and maintains soil fertility over time. The platform's modular design also makes it extendable, enabling future integration with IoT-based soil sensors, mobile app extensions, QR-based farm identification, and AI-driven yield prediction models.

Overall, the Soil Testing Web Application bridges the gap between traditional farming practices and modern technology. It empowers users to adopt scientific, data-driven approaches to soil management, promotes environmental conservation, and contributes to the growth of sustainable agriculture. By offering an accessible alternative to costly laboratory testing, the system proves to be a valuable tool for farmers, gardeners, agricultural students, and rural communities aiming to improve productivity and soil health in an efficient and affordable way.

INDEX

CHAPTER 1: INTRODUCTION 10
1.1 Background 10
1.2 Objectives 14
1.3 Purpose, Scope, and Applicability 14
1.3.3 Applicability 16
1.4 Achievements 17
1.5 Organization of Report 19
CHAPTER 2: SURVEY OF TECHNOLOGIES 22
CHAPTER 3: REQUIREMENTS AND ANALYSIS 32
3.1 Problem Definition 32
3.2 Requirements Specification 34
3.3 Planning & Scheduling 37
3.4 Software and Hardware Requirements 40
CHAPTER 4: SYSTEM DESIGN 42
4.1 UML Diagrams 44
4.1.1 Use Case Diagram 45
4.1.2. Class Diagram 46
4.1.3 Entity Relationship (ER) Diagram 48
4.1.4 System Flowchart 49
4.2 Module Design 51
4.3 Database Design 54
4.3.1 Schema Design 54
4.3.2 Data Integrity and Constraints 57
4.3.3 Procedural Design 57
4.3.4 Data Structures Used 58
4.3.5 Algorithm Design 60
CHAPTER 5: IMPLEMENTATION AND TESTING 62
5.1 Implementation Approaches 62
5.2 Overview of the Implementation 63
5.3 Development Tools and Technology Stack 64
5.4 System Architecture 66
5.4 Module-wise Implementation 68
5.4.1 User Interface Design 69
5.4.2 Functional Modules 69

5.4.3 Database Integration 70
5.4.4 Input/Output Handling 71
5.5 Challenges Faced and Solutions Implemented 71
5.6 Testing Strategy and Methodology 73
5.6.1 Unit Testing 74
5.6.2 Integration Testing 74
5.6.3 System Testing 75
5.6.4 Test Cases with Expected and Actual Results 76
CHAPTER 6: RESULTS 85
6.1 Test Reports 85
6.1.1 Discussion of Test Results 86
6.2 User Documentation 88
CHAPTER 7: CONCLUSIONS 94
7.1 Significance of the System 94
7.2 Limitations of the System 94
7.3 Future Scope of the Project 95
7.4 Practical Implications 96
CHAPTER 8: REFERENCES / BIBLIOGRAPHY 97
CHAPTER 9 : ANNEXURE 98

CHAPTER 1: INTRODUCTION

1.1 Background

Soil is one of the most valuable, fundamental, and irreplaceable natural resources on Earth. It forms the very foundation of all terrestrial life and serves as the backbone of agricultural production, ecological balance, and global food systems. Acting as much more than a medium for plant growth, soil performs crucial functions such as regulating water flow, supporting biodiversity, recycling nutrients, detoxifying pollutants, and storing carbon. Its composition—an intricate blend of minerals, organic matter, microorganisms, water, and air—determines its ability to support vegetation and sustain biological processes. The quality of soil directly influences agricultural productivity, food availability, nutritional quality of crops, and long-term environmental sustainability. Healthy and fertile soil is characterized by a balanced mix of nutrients, stable organic matter, beneficial microorganisms, adequate moisture retention, and suitable physical structure that promotes root penetration and nutrient absorption. These characteristics are vital for supporting plant growth and ensuring high-quality crop yields.

However, over the past few decades, rapid agricultural expansion, population growth, industrialization, and climate change have placed increasing pressure on soil quality. The global demand for food has led to intensified farming practices that often overlook the importance of long-term soil conservation. Modern agriculture commonly relies on chemical fertilizers, pesticides, insecticides, and continuous cropping cycles to meet increasing production demands. While these methods may provide short-term gains, excessive and unbalanced use of such inputs results in long-term soil degradation. Key issues such as nutrient depletion, soil acidification, salinity buildup, erosion, compaction, and loss of organic matter have become widespread challenges. Degraded soil not only produces lower yields but also becomes less resilient to environmental stresses such as drought, heavy rainfall, and temperature fluctuations.

Climate change further accelerates soil degradation. Erratic rainfall patterns lead to increased erosion and nutrient leaching, while prolonged drought reduces organic matter and microbial activity. Rising temperatures speed up decomposition rates and disrupt soil moisture balance. Floods wash away topsoil—the most fertile layer rich in nutrients and microorganisms. Because of these combined factors, farmers in many regions face declining soil fertility, unpredictable crop yields, and increasing dependency on external chemical inputs simply to maintain previous productivity levels. As soil health continues to deteriorate, understanding soil conditions and adopting appropriate soil management strategies becomes essential for sustaining agricultural livelihoods.

Limitations of Traditional Soil Testing Methods

Historically, soil testing has been carried out through laboratory-based analysis. Farmers collect samples from their fields, dry and pack them, and then send them to government or private laboratories for scientific testing. These laboratories measure various parameters such as pH, nitrogen (N), phosphorus (P), potassium (K), micronutrients like zinc and iron, organic carbon levels, moisture content, and soil texture. While laboratory testing remains the most accurate method, it comes

with several limitations that reduce its accessibility and effectiveness for many farmers:

- Time-Consuming Process
  Laboratory testing can take several days or even weeks, especially during busy agricultural seasons when thousands of samples are submitted simultaneously. Delays in receiving results prevent farmers from making timely decisions regarding fertilizer application, irrigation planning, and crop selection. Since agriculture is highly season-dependent, delays can significantly affect productivity.
- High Cost and Accessibility Barriers
  Regular soil testing is often too expensive for small and marginal farmers, who form the majority of the agricultural population in developing countries. Many rural farmers also live far from testing laboratories, which forces them to travel long distances and spend additional money on transportation. These financial and logistical constraints discourage routine soil testing.
- Lack of Awareness and Understanding
  Even when farmers do receive laboratory reports, many struggle to interpret the scientific terminology and technical charts included in them. Without proper guidance, they find it difficult to convert the data into actionable farming decisions. As a result, they may continue relying on traditional practices or guesswork rather than making scientifically informed choices.
- Labor-Intensive and Inconvenient
  The process of collecting samples, drying them properly, packing them in appropriate containers, and delivering them to labs requires significant effort. Many farmers lack the time or resources for such repeated testing procedures. Consequently, soil analysis becomes an infrequent activity rather than a regular part of farm management.

Because of these limitations, only a small percentage of farmers consistently test their soil. Most base their fertilizer application and crop decisions on experience passed down through generations, which often leads to imbalanced nutrient application, unnecessary soil degradation, and reduced yields. Without reliable soil health data, precision agriculture practices—which optimize input usage—cannot be effectively adopted.

Emergence of Smart Farming and Digital Agriculture

To overcome traditional limitations, modern agriculture is increasingly embracing digital solutions. The rise of smart farming, supported by advances in computer science, artificial intelligence, embedded systems, and mobile technology, has transformed agricultural practices. Smart farming tools such as IoT sensors, mobile-based advisory systems, satellite imagery, expert systems, and decision-support platforms are being used to gather real-time agricultural data and deliver timely recommendations.

Digital agriculture aims to make farming more efficient, sustainable, and profitable. It helps farmers automate data collection, monitor soil and crop health, and make informed decisions. Several attempts have been made to modernize soil evaluation using digital tools like mobile apps, portable soil testing kits, sensor-based devices, and AI-driven analysis platforms. These solutions provide faster results and greater accessibility but still have limitations:

- Many apps offer only simple pH measurements and lack comprehensive nutrient analysis.
- Some provide generic recommendations that do not consider soil type, moisture content, environmental conditions, or specific crop requirements.
- Existing tools often fail to integrate multiple soil parameters into one cohesive system.
- Many digital platforms do not store user history, making it difficult to track soil improvements over time.
- Personalized agricultural advice is often missing; instead, generalized suggestions are provided.
- Downloadable professional soil health reports are rarely available.

Despite these innovations, there is still a significant gap in providing farmers with a complete, integrated, user-friendly, and scientifically reliable digital soil assessment system.

Need for a Digitized Soil Testing Platform

Given the increasing complexity of modern agriculture, the limitations of traditional soil testing methods, and the shortcomings of existing digital tools, there is a clear need for an advanced, accessible, and intelligent soil testing platform. A well-designed digital soil analysis system can empower farmers, home gardeners, agricultural extension workers, and researchers by providing instant, accurate, and easy-to-understand soil health insights.

Such a platform can:

- Reduce reliance on expensive laboratory tests.
- Enhance farmer awareness and promote scientific decision-making.
- Encourage frequent soil monitoring, improving long-term soil health.
- Provide tailored recommendations instead of generalized advice.
- Support sustainable agriculture through optimized input usage.

An effective digital tool can simplify soil analysis, improve agricultural management, and contribute to environmental conservation and economic stability.

Proposed Solution: Soil Testing Web Application

To address these challenges, this project proposes a Soil Testing Web Application—a user-friendly, modern, and cost-effective platform designed to make soil analysis accessible to everyone. The system allows users to manually enter soil parameters or integrate data from sensors. The platform then analyzes this data using scientifically validated agricultural guidelines and provides personalized recommendations.

The application accepts inputs such as:

- pH Levels: Indicates whether soil is acidic, neutral, or alkaline.
- NPK Values: Provides macronutrient content essential for plant growth.
- Moisture Percentage: Helps determine irrigation needs.
- Soil Type: Includes clayey, loamy, sandy, and mixed textures.
- Environmental Conditions**:** Manually entered or fetched automatically.

Based on these inputs, the system provides:

1.Detailed Soil Health Interpretation

Clear explanations of nutrient deficiencies or excesses, pH suitability, moisture conditions, and soil structure.

2. Personalized Crop Recommendations

Crops suitable for the user's soil type, nutrient profile, and local climate conditions.

3. Fertilizer and Soil Treatment Plans
Guidance on:
- Fertilizer type and quantity
- Soil amendments (lime, gypsum, organic matter, compost)
- Sustainable practices for long-term soil health

4. Data Storage and History Tracking
Enables users to monitor changes, compare past test results, and track long-term soil health improvements.
5. Downloadable PDF Reports
Clean, structured reports for documentation, farming consultations, or agricultural planning.

Research and Technological Foundation
The project draws from established research in soil science, environmental studies, data analysis, and digital agriculture. Studies emphasize the need to integrate multiple soil parameters for accurate assessment rather than relying on single factors like pH or moisture alone. Research on decision-support systems shows that user-friendly digital interfaces significantly improve farmer adoption of scientific practices.
The web application integrates knowledge from:
- Agricultural Science: Understanding soil chemistry, nutrient cycles, crop nutrient requirements.
- Computer Science: Including database management, algorithm development, and UI/UX design.
- Expert Systems: Rule-based decision-making models for interpreting soil parameters.
- Web Development: Technologies such as HTML, CSS, JavaScript, backend frameworks, and cloud-based architectures.

This interdisciplinary approach makes the system robust, modern, and practical for real-world agricultural use.

Impact and Significance
The Soil Testing Web Application has the potential to significantly transform soil management practices. Its benefits include:
- Higher Productivity: Matching crops to suitable soil conditions increases yield.
- Cost Efficiency: Reduces unnecessary spending on fertilizers and chemicals.
- Environmental Protection: Helps prevent overuse of fertilizers and reduces soil pollution.
- Accessibility: Easy to use for small-scale farmers, students, and gardeners.
- Empowerment: Provides scientific knowledge in clear and understandable form.

In the long term, such tools contribute to:
- Sustainable farming systems
- Improved food security
- Better soil conservation
- Climate-resilient agriculture
- Enhanced digital literacy in rural areas

By combining scientific accuracy, technological innovation, and user-friendly design, this digital platform supports both immediate agricultural needs and long-term environmental sustainability.

1.2 Objectives

1. To develop a web-based system that analyzes soil parameters such as pH, moisture, and NPK levels.
2. To generate accurate crop recommendations and fertilizer guidance based on soil quality.
3. To provide users with soil improvement suggestions and detailed soil health reports.
4. To maintain a history of soil tests, enabling users to track changes and monitor long-term soil health.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The primary purpose of the Soil Testing Web Application is to bridge the persistent technological and informational gap that exists within the agricultural sector, particularly among small-scale and rural farmers who lack access to timely and affordable soil testing services. Agriculture remains heavily dependent on soil quality, yet millions of farmers across developing regions continue to rely on guesswork or traditional knowledge when selecting crops or applying fertilizers. This often leads to nutrient imbalance, reduced productivity, increased expenses, and long-term soil degradation. The Soil Testing Web Application aims to address these challenges by offering an innovative, cost-effective, and easily accessible digital platform that converts complex soil science into simple, actionable insights.

Through a user-friendly interface, the system enables users to enter key soil parameters such as pH, moisture content, and macronutrient levels—nitrogen (N), phosphorus (P), and potassium (K). These parameters are essential for determining soil health and making informed agricultural decisions. Instead of depending on laboratory results that can take days or weeks, the application provides instantaneous analysis and expert-level recommendations. The platform serves as a virtual agricultural advisor, helping users understand the implications of their soil data, identify nutrient deficiencies or excesses, and choose corrective measures to improve soil fertility.

Another important purpose of the application is to discourage unscientific farming practices that contribute to soil degradation. Overuse of chemical fertilizers, uninformed selection of crops, and lack of awareness about soil conditions often result in declining soil productivity. By offering precise fertilizer recommendations and soil treatment guidelines, the platform promotes sustainable farming and encourages the use of balanced nutrient management practices. This intellectual empowerment helps farmers reduce production costs, avoid unnecessary environmental pollution, and maintain long-term soil health.

Furthermore, the application integrates weather-related information and environmental factors, allowing users to consider climatic conditions before making decisions. Weather greatly influences soil behavior; therefore, integrating such data enables farmers to identify appropriate planting seasons, forecast irrigation needs, and adopt climate-resilient farming strategies. This feature ensures that decisions are not only soil-based but also environmentally relevant.

Beyond agricultural benefits, one broader purpose of the project is to demonstrate the power of digital technologies in solving real-world challenges. The system showcases how computer science concepts—such as data processing, algorithm design, database management, and interface design—can contribute to environmental sustainability, climate resilience, and community development. By making the application simple enough for non-technical users, the project also promotes digital literacy among rural populations. Overall, the purpose extends beyond soil analysis to contribute toward sustainable development goals, strengthen local agricultural ecosystems, and pave the way for precision agriculture in regions where technological adoption is generally low.

1.3.2 Scope

The scope of the Soil Testing Web Application encompasses the development of a complete digital ecosystem aimed at facilitating quick, accessible, and scientifically reliable soil evaluation. The project covers the full software development lifecycle, beginning with requirement analysis, interface design, and system architecture planning, and extending through backend logic implementation, database configuration, integration with third-party APIs, deployment, and real-world testing.

At the core of the system is the soil nutrient analysis module, which interprets user-provided data using standardized agricultural benchmarks. These benchmarks are derived from research-based nutrient thresholds and pH categorizations used in modern soil science. The system processes values for nitrogen, phosphorus, potassium, soil moisture, and pH to generate meaningful insights. Decision-making logic is implemented using condition-based rules and scientific guidelines, ensuring that the output is both accurate and actionable.

Frontend development includes creating a responsive, intuitive, and visually clean user interface that supports multiple device types, including smartphones, tablets, and desktop computers. This is essential because farmers may not always have access to advanced devices or high-speed internet. The interface is designed to minimize input complexity while maximizing clarity of results, ensuring accessibility for users with limited technological skills.

Backend development involves implementing algorithms for soil interpretation, crop recommendation, fertilizer calculation, and soil treatment guidance. Database management includes creating tables for soil entries, user profiles, historical records, and crop databases. Storing user history allows the system to provide trend analysis and helps users monitor improvements over time. Report generation is another critical component; the system produces downloadable PDF or printable reports summarizing soil health insights, recommendations, and historical comparisons.

The project also includes integration with external weather APIs to provide temperature, rainfall, and climate information relevant to the user's location. This allows the web application to provide more holistic agricultural advice. However, there are certain defined assumptions and limitations within the scope. The system assumes that users will input accurate soil parameter values obtained through basic kits or sensors. It does not validate data scientifically or physically verify soil conditions. The application does not cover advanced artificial intelligence technologies such as machine learning–based prediction models, nor does it include IoT-based automatic soil monitoring systems. However, the architecture has been designed in a modular way to

allow future enhancements such as AI-based crop forecasting, automated irrigation systems, pest detection features, or integration with government agricultural platforms.

Overall, the project focuses on the core functionality of soil analysis, user guidance, environmental awareness, and digital accessibility. While it does not attempt to replace laboratories entirely, it offers a practical alternative for preliminary soil assessment and everyday decision-making.

1.3.3 Applicability

The Soil Testing Web Application has a wide range of both direct and indirect applications. Its adaptability and user-friendly design make it suitable for farmers, students, researchers, agricultural staff, environmental workers, and even home gardeners. The following sections detail its applicability in various real-world scenarios:

Direct Applications

1. Agricultural Decision-Making

The most immediate application of the system is to support farmers in making scientifically informed decisions. By providing quick insights into soil nutrient levels and chemical balance, the system helps farmers choose appropriate crops, determine fertilizer quantity, and apply necessary soil treatments. This minimizes the guesswork associated with traditional farming and helps farmers maximize yield while reducing unnecessary expenses. Whether the farmer is selecting seasonal crops or attempting to restore degraded soil, the platform acts as a decision-support tool.

2. Home Gardening and Urban Farming

The rise of urban agriculture, rooftop gardens, balcony plantations, and community gardens has created a need for simple tools that guide non-experts. Home gardeners often face challenges in identifying soil suitability for specific plants. The platform provides clarity by allowing them to enter basic soil parameters and receive personalized plant recommendations, thereby improving plant survival rates and promoting healthier, pesticide-free home-grown produce.

3. Academic and Educational Usage

Students studying agriculture, environmental science, botany, geography, or data science can use the platform as a practical learning tool. By experimenting with various soil inputs, they can observe how nutrient compositions influence plant growth. This real-time feedback strengthens theoretical knowledge and enhances understanding of soil science concepts that are typically difficult to visualize.

4. Research and Field Analysis

Researchers studying soil quality trends, climate impacts, nutrient mobility, or agricultural systems can use the platform to collect, record, and analyze soil data. The ability to store and compare historical results provides valuable insights into seasonal variations, soil degradation patterns, or recovery effectiveness after using organic treatments.

Indirect Applications

1. Environmental Sustainability

The application indirectly contributes to environmental sustainability by promoting balanced fertilizer usage. Overuse of nitrogen, phosphorus, and potassium fertilizers leads to soil pollution, water contamination, and reduced biodiversity. By offering precise recommendations, the platform helps prevent nutrient runoff, supports soil conservation, and fosters long-term ecological balance.

2. Adoption of Digital Agriculture

The app encourages farmers—especially those in rural areas—to use digital tools in their daily agricultural practices. This exposure not only improves their productivity but also builds confidence in adopting advanced technologies such as IoT sensors, mobile advisory systems, and data-driven farming. The system acts as a stepping stone toward smart farming techniques.

3. Foundation for Future Advanced Systems

The modular structure of the project allows it to serve as a foundation for advanced agricultural technologies. Future enhancements such as machine learning–based prediction, pest detection modules, automated irrigation scheduling, or integration with weather-based advisory systems can be easily incorporated. Thus, the platform supports long-term innovation in agri-tech.

4. Educational Workshops and Community Training

Teachers, NGO workers, and agricultural trainers can use the system during hands-on training programs or community workshops. The platform simplifies scientific concepts and makes it easier to demonstrate soil testing principles to groups with varying educational backgrounds.

5. Government and Community-Based Programs

Local agricultural departments, farmers' cooperatives, or government agencies aiming to promote soil health awareness can integrate this platform into their community programs. It helps in gathering soil data at a large scale, identifying regional nutrient deficiencies, and formulating soil health management plans.

6. Technological Skill Development

For computer science students and developers, the project serves as an excellent case study demonstrating how web development, database integration, and environmental science can work together. It highlights interdisciplinary problem-solving and showcases how digital tools can address agricultural challenges.

1.4 Achievements

The completion of the Soil Testing Web Application marks a significant accomplishment in both academic learning and practical technical development. Throughout the stages of analysis, design, implementation, and testing, numerous achievements were realized. These can be categorized into knowledge and skills gained, technical contributions, and successful attainment of project goals.

1.4.1 Knowledge and Skills Gained

- Full-Stack Development Skills:
  The project enhanced proficiency in web development technologies such as HTML, CSS, JavaScript for the frontend, and Python/Flask or Node.js for backend logic. Students gained hands-on experience with API integration, database management, routing, and server-side processing.
- Database Design and Data Handling:
  Working with soil test values required structured database creation using SQL or NoSQL systems. Students learned efficient data storage, retrieval, validation, and record management—skills essential for any professional IT environment.
- Algorithmic Thinking and Decision Logic:
  Implementing soil analysis algorithms—such as nutrient deficiency evaluation, pH balancing logic, and fertilizer recommendation rules—improved students' problem-solving, logical structuring, and analytical skills.
- Domain Knowledge in Soil Science:
  Through the project, students gained foundational awareness of soil nutrients (NPK), pH range behavior, moisture importance, micro-nutrient roles, and how these factors influence plant growth, helping bridge the gap between technology and agriculture.
- User Interface and UX Design:
  Building a clean, responsive interface strengthened skills in layout design, color balancing, accessibility, and usability testing. Students learned how to design dashboards, forms, result pages, and recommendation cards that are simple and user-friendly.
- Software Engineering Practices:
  The project provided practical experience with SDLC phases, including requirement gathering, design diagrams, coding standards, debugging, test planning, and documentation. This strengthened understanding of structured and professional development processes.

1.4.2 Contributions to the Chosen Area

- Educational Contribution:
  The project demonstrates how software can support agriculture by converting scientific soil parameters into actionable advice. It serves as a reference project for students interested in agritech, environmental IT solutions, and real-world data processing systems.
- Promotion of Digital Agriculture:
  The application shows how traditional farming methods can be enhanced using modern technology. It encourages farmers and students to adopt digital tools that simplify soil health assessment and crop planning.
- Practical Application of WebTechnologies:
  The system highlights how web technologies can be used to build decision-support applications. It acts as a valuable example for academic institutions teaching full-stack development and applied computing.
- Low-Cost Alternative for Communities:
  By offering a software-based analysis method, it supports small farmers and gardeners who may not always have access to expensive lab testing. This aligns with social impact goals in the agricultural domain.

- Foundation for Future Enhancements:
  The project provides a strong basis for advanced modules like machine learning predictions, IoT-based soil sensors, real-time weather-based crop suggestions, and automated fertilizer scheduling systems.

1.4.3 Goals Achieved

At the beginning of the project, several objectives were defined. The project successfully met—and in some areas exceeded—these goals:

- To analyze soil parameters such as pH, nitrogen, phosphorus, potassium, and moisture.
  Achieved. The system accurately processes user inputs and classifies soil health.
- To recommend suitable crops based on soil characteristics.
  Achieved. The application dynamically suggests crops that match soil nutrient levels and pH conditions.
- To provide fertilizer recommendations to correct deficiencies.
  Achieved. The system generates NPK-specific fertilizer suggestions and improvement tips.
- To maintain a history of soil tests for individual users.
  Achieved. All tests are stored securely and can be revisited through the user dashboard.

Additional goals were achieved beyond the original scope:

- Integration of weather information for more accurate recommendations.
- Generation of detailed downloadable reports for each soil test.
- Enhanced UI optimization for mobile and desktop responsiveness.

1.4.4 Overall Significance of Achievements

The achievements of the Soil Testing Web Application extend far beyond the technical outcomes. Academically, students strengthened their understanding of full-stack development, data handling, and analytical problem-solving. Professionally, the skills gained can be applied in careers related to software development, data-driven systems, agritech, and environmental IT solutions.

The project's broader significance can be summarized as follows:

1. Strengthening academic learning through hands-on practical implementation.
2. Enhancing technical expertise in web development and algorithm design.
3. Encouraging innovative thinking in agricultural technology.
4. Promoting sustainable farming practices and supporting community welfare.
5. Providing a scalable platform for further research, automation, and AI-based advancements.

1.5 Organization of Report

The remaining chapters of this project report are structured in a systematic manner to provide a clear understanding of the development process, methodologies used, and outcomes achieved through the Soil Testing Web Application.

Chapter 1: Introduction

This chapter lays the foundation for the entire project. It covers the background and need for a digital soil testing solution, the problem statement, the aims and objectives, the purpose and scope of the system, and the applicability of the project in real-world scenarios. It also highlights the significance of using technology to support modern agriculture.

Chapter 2: Literature Survey

This chapter explores existing soil testing methods—both manual and laboratory-based—as well as current digital tools used in agriculture. It reviews research papers, online tools, mobile apps, and government soil health platforms. A comparative study is presented to identify gaps in current solutions and justify the need for the proposed system.

Chapter 3: System Analysis

Chapter 3 provides an in-depth analysis of the project requirements. It includes functional and non-functional requirements, feasibility studies (technical, operational, economic, and schedule feasibility), and a detailed analysis of the current problems faced by farmers and gardeners. The proposed system's features and advantages over existing solutions are also discussed.

Chapter 4: System Design

This chapter translates system requirements into technical design elements. It covers the overall system architecture, flow of data, data flow diagrams (Level 0, 1, and 2), entity–relationship (ER) diagram, database schema design, and module explanations. User interface design sketches, navigation flowcharts, and component diagrams are also presented to illustrate system structure.

Chapter 5: Implementation

Chapter 5 explains how the designed system is converted into working software. It describes the development environment, programming languages, frameworks, and technologies used. The chapter includes backend logic for soil analysis, frontend interface development, server configuration, form validations, and database integration. Screenshots and code snippets of important modules may be provided for clarity.

Chapter 6: Testing and Results

This chapter focuses on validating the system. It includes various test cases used to ensure functionality, usability, reliability, and accuracy of soil recommendations. Input/output samples, error handling, and result interpretations are presented. The chapter also evaluates whether the system meets its intended goals and compares results with expected outcomes.

Chapter 7: Conclusion and Future Enhancements

The final chapter summarizes the findings and achievements of the project. It reflects on the success of the system in meeting its objectives and highlights the overall impact of the solution. Future enhancement possibilities—such as

integrating IoT-based soil sensors, using machine learning for crop prediction, adding multilingual support, and developing a mobile app version—are discussed.

CHAPTER 2: SURVEY OF TECHNOLOGIES

Moonrungsee, Pencharee, and Jakmunee (2015) pioneered a highly practical mobile-phone-based colorimetric phosphorus analysis system, establishing a foundation for transforming smartphones into capable, field-ready analytical instruments. Their study focused on the determination of available phosphorus in soil using molybdenum blue chemistry, a method widely used in professional laboratories. The innovation lay not in the chemistry itself but in how the researchers redesigned the measurement process to work reliably on a standard Android smartphone. They constructed a carefully controlled imaging environment using a light-tight measurement box fitted with white LEDs to eliminate inconsistencies caused by ambient lighting. The phone camera captured images of soil-reacted samples, and RGB values extracted from these images were mathematically correlated with phosphorus concentration. By comparing results with those obtained from laboratory spectrophotometers, the team demonstrated excellent agreement, proving that smartphone-based soil assays can rival conventional lab methods when proper calibration and lighting control are applied.
This study has profound implications for designing a modern soil-testing mobile application. First, it demonstrates that nutrient analysis—traditionally restricted to laboratory settings—can be decentralized and made accessible to non-specialists such as farmers and field technicians. The low-cost hardware used in this research (simple reagents, LEDs, and a smartphone camera) presents a scalable pathway for widespread deployment, especially in developing regions where access to laboratory facilities is limited. The researchers also emphasized the importance of standardized image capture protocols, including fixed illumination, stable sample positioning, and controlled reaction times. These principles can be fully integrated into a mobile app by providing step-by-step guidance, visual instructions, and countdown timers to ensure uniform testing conditions across diverse users.
The concept of a portable "light-tight box" offers important design insights. While the original prototype used a rigid structure, a soil-testing app can recommend more user-friendly adaptations, such as foldable cardboard imaging chambers or lightweight clip-on LED attachments designed for smartphones. The app can also incorporate digital calibration routines to adjust for differences between phone camera sensors, white balance settings, and varying exposure levels. By implementing these correction models, the system can maintain high measurement accuracy across a wide range of smartphone brands and models. This flexibility is crucial for real-world usage, where smartphones differ widely in hardware quality.
Beyond phosphorus analysis, the work by Moonrungsee et al. provides a versatile technological blueprint for extending smartphone colorimetry to other soil nutrients. With modified reagent chemistries, the same imaging and processing pipeline can be applied to nitrogen species, potassium, micronutrients (such as zinc and iron), and soil contaminants. Their validation approach—comparing smartphone readings with laboratory spectrophotometric data—can be replicated for each nutrient to ensure scientific reliability. This ensures that the app does not merely approximate values but provides trustworthy measurements supported by empirical calibration.

In terms of broader system architecture, this research supports a hybrid app design that merges chemical testing, smartphone imaging, algorithmic correction, and user guidance into one unified platform. The ability to convert digital color signals into

quantitative nutrient data empowers users to generate actionable insights without technical expertise. Ultimately, Moonrungsee et al.'s study serves as a cornerstone for the soil-testing app, proving that smartphone-based analytical chemistry is not only feasible but capable of delivering laboratory-grade accuracy when supported by proper calibration, environmental control, and robust image-processing algorithms.

Ait Errayess et al. (2018) developed a smartphone-based colorimetric sensing platform designed for accurate field analysis of chemical substances, demonstrating how mobile devices can serve as robust analytical instruments when paired with well-controlled hardware and software environments. Their study emphasized that a smartphone's camera—despite differences in quality across brands—can reliably quantify colorimetric reactions when standardized capture conditions are applied. By designing a compact, controlled lighting module and applying advanced image-processing techniques, they showed that smartphone-assisted colorimetry can achieve laboratory-grade measurement accuracy for several analytes. The researchers addressed challenges such as ambient light variability, camera sensor inconsistencies, and color distortion by incorporating color-correction algorithms, background normalization, and controlled LED illumination. Their findings highlight that the smartphone is not merely a data recorder but a significantly capable, portable chemical analyzer.

For a soil-testing app, this work provides essential design principles. A soil-testing mobile platform can instruct users to capture soil-test color reactions under specified conditions—such as using a foldable black-box chamber or a clip-on LED module—similar to the setup used in the study. The app can apply built-in color-correction algorithms, white-balance normalization, and RGB/HSV extraction methods to ensure consistent performance across different phone models. These techniques help address the unavoidable diversity in user devices and testing environments. The study also underscores the importance of providing users with step-by-step guidance that reduces human error, such as instructions on reagent mixing, reaction timing, and camera alignment.

The researchers further demonstrated that accurate colorimetric sensing depends heavily on software workflows, including noise reduction, histogram normalization, and perspective correction. A soil-testing app can integrate similar routines to automatically detect the test area, analyze the color intensity, and avoid user mistakes such as angled images, blurry captures, or shadow interference. This ensures that even non-technical users can obtain reliable results. Additionally, the study's modular design indicates that soil nutrient kits—like nitrate, phosphorus, potassium, and iron tests—can be extended without redesigning the entire system. Only reagent chemistry and calibration curves need modification.

A key takeaway from Ait Errayess et al.'s research is the strong emphasis on reproducibility and environmental control. Their approach validates that smartphone-based sensors can achieve accurate chemical detection if supported by structured hardware, standardized procedures, and intelligent software. For a soil-testing app, these insights reinforce the need for a controlled capture interface, guided instructions, and robust algorithmic correction. Ultimately, this study strengthens the feasibility of using smartphones as central devices for soil analysis

and provides a clear blueprint for building a scientifically sound mobile testing platform.

Firdaus et al. (2019) presented a significant advancement in the development of portable, low-cost analytical tools by demonstrating the integration of smartphone imaging technology with paper-based microfluidic analytical devices (µPADs). Their research showed how colorimetric reactions performed on paper strips can be accurately quantified using smartphone cameras, thereby eliminating the need for bulky laboratory instruments. The study highlighted several key elements that make this approach particularly suitable for field applications: the simplicity of fabrication of test papers, the portability and durability of µPADs, and the inherent accessibility of smartphones. The core principle involves depositing chemical reagents onto patterned paper channels, applying soil extract samples, and allowing a visible color change to occur. The smartphone captures an image of the reacted strip, and RGB or HSV values are extracted and processed to determine the concentration of the analyte. Firdaus and colleagues emphasized that with proper calibration and environmental control, this method can produce reliable, semi-quantitative results comparable to more sophisticated analytical methods.

This study directly influences the conceptual design of a soil-testing mobile application. It provides a template for using paper-based test strips as one of the primary input mechanisms for soil nutrient, pH, and salinity measurement. Because the paper strips are lightweight, inexpensive, and easy to transport, farmers and field workers can perform a wide range of soil tests without specialized knowledge or equipment. The smartphone app can guide users through the testing workflow—wetting the sample, applying it to the paper, waiting for the reaction time to complete, and capturing the image under recommended lighting conditions. Firdaus et al. stressed the importance of consistent illumination and camera positioning, which can be incorporated into the app via visual overlays, on-screen alignment boxes, and auto-exposure locking features. These app-based controls help ensure that data quality remains uniform across diverse testing environments.

Another major contribution of the study is the discussion on calibration and standardization. The researchers highlighted that variations in smartphone cameras, paper material quality, and ambient lighting can all influence the measured color intensity. To address this, they suggested using reference calibration cards or embedding color reference patches on the test strips themselves. A soil-testing app can adopt this strategy by prompting users to photograph the strip alongside a built-in color reference region. Machine-learning-based color correction algorithms can further compensate for device-to-device variability, ensuring that results remain highly accurate regardless of phone model or lighting conditions. This enables the system to scale to millions of users with minimal hardware dependency.

The study also identified the practical challenges associated with field deployment, including humidity sensitivity of paper devices, potential contamination during handling, and the need for proper timing and reagent storage. These insights help shape app features such as built-in reaction timers, step-by-step tutorials, and notifications for maintaining test strip quality. Additionally, the simplicity of paper-based devices makes them compatible with a subscription-based consumable model, where users periodically purchase new soil nutrient strips supported by the app's analytical backend.

Firdaus et al.'s research ultimately demonstrates that combining paper-based analytical devices with smartphone imaging offers a powerful, scalable, and cost-effective approach to soil testing. It allows users—particularly those in rural or resource-limited regions—to access chemical analysis tools that would otherwise be financially or logistically inaccessible. For a soil-testing mobile application, the principles from this paper support the integration of paper-based nutrient tests, image-processing algorithms, calibration workflows, and guided user interfaces, forming a cornerstone of an inclusive, user-friendly, and scientifically reliable soil-analysis platform.

Baranwal et al. (2022) presented a comprehensive review of electrochemical sensors, focusing on their working principles, performance metrics, and applications across environmental and agricultural monitoring. The review explained the operating mechanisms of amperometric, potentiometric, and impedimetric sensors, offering detailed insights into how these low-cost, high-sensitivity devices detect ions and analytes in real samples. The authors highlighted the growing use of electrochemical probes for nutrient estimation, such as nitrate, ammonium, potassium, pH, and electrical conductivity (EC), noting their speed, portability, and ability to produce continuous, real-time measurements. They also discussed the limitations associated with electrode fouling, drift, matrix effects, and calibration requirements—issues that are especially relevant for soil environments where heterogeneity can affect readings.

This research provides valuable guidance for a soil-testing mobile application that aims to support optional hardware accessories. Electrochemical sensors, such as ion-selective electrodes (ISEs) or multiparameter soil probes, can connect to the app via Bluetooth or USB interfaces. The app can serve as a central hub for data acquisition, logging, calibration reminders, and interpretation. Baranwal et al.'s emphasis on calibration procedures—such as two-point calibration for pH or multi-standard calibration for nitrate—guides the development of app-based calibration workflows with visual instruction prompts. Their explanation of signal drift and soil-matrix effects reinforces the need for real-time correction algorithms or AI models that adjust readings based on historic sensor behavior.

The review also highlighted that electrochemical sensors are increasingly being integrated into IoT systems for continuous agricultural monitoring. A soil-testing app can incorporate IoT connectivity for advanced users, enabling real-time dashboards, long-term soil-health trends, and alerts when nutrient levels fall below optimal thresholds. However, Baranwal et al. also warned that such systems require stable power sources, protective casings, and regular maintenance—practical aspects the app should communicate clearly to users.

The authors identified a key advantage of electrochemical sensors: they can directly quantify nutrient ions without requiring laboratory reagents, making them suitable for rapid field testing. Integrating such sensors into the app as optional add-ons allows the platform to serve both low-budget farmers and advanced users like agronomists, researchers, and commercial farming operations. Ultimately, the review validates electrochemical sensing as a technically strong and scalable method for soil analysis while informing the app's design considerations related to calibration, maintenance, data recording, user training, and sensor interoperability.

Kim et al. (2022) conducted a systematic review focusing on electrochemical sensors adapted for precision agriculture applications, particularly those designed to monitor soil nutrients, water quality, and plant health. The authors examined miniaturized probe technologies, wearable sensors for plants, and embedded soil-sensor arrays capable of long-term operation. The study highlighted their advantages—high sensitivity, rapid response, real-time monitoring capacity, and suitability for both indoor and outdoor agricultural environments. These sensors allow for more accurate nutrient management strategies, enabling farmers to make timely decisions that can significantly improve crop yield and resource efficiency.

From the perspective of a soil-testing app, the findings in Kim et al.'s research support the inclusion of an advanced "sensor integration" module. The app can connect to electrochemical probes through Bluetooth or Wi-Fi, retrieving real-time measurements of nitrate, ammonium, potassium, pH, soil moisture, and EC. The review also noted that many of these sensors require routine recalibration and can be influenced by soil temperature, salinity, and organic content. To address this, the app can incorporate guided workflows that remind users to perform calibration at scheduled intervals, detect anomalies, and apply correction factors based on environmental metadata.

The researchers also explored deployment challenges such as sensor longevity, soil fouling, and cost constraints. While miniaturized sensors are becoming affordable, they may still be expensive for small-scale farmers. A soil-testing app can help mitigate this by offering layered functionality—basic features for users relying solely on the smartphone's camera and advanced features for those who purchase compatible sensor hardware. Kim et al. also emphasized that electrochemical sensors have the potential to deliver high-resolution spatial and temporal data, making them ideal for integration with machine learning models that can predict nutrient dynamics or irrigation needs.

This study demonstrates that sensor-based soil monitoring significantly enhances accuracy compared to manual, snapshot-based testing. By incorporating insights from Kim et al., the soil-testing app can support high-end agricultural stakeholders while remaining accessible to general users. The study reinforces the value of hybrid systems where smartphone-based testing and sensor-based data collection co-exist, offering flexible, scalable, and advanced agricultural monitoring solutions.

Abdulraheem et al. (2023) provided an extensive review of remote sensing techniques used for soil property estimation, covering passive and active sensors including multispectral, hyperspectral, thermal, SAR (Synthetic Aperture Radar), and LiDAR. They examined how spectral reflectance patterns and microwave backscatter signatures correlate with soil attributes such as organic carbon, salinity, moisture, surface texture, and mineral composition. The study emphasized that remote sensing allows large-scale soil mapping and long-term monitoring, providing valuable insights at field, regional, and national scales.

For a soil-testing mobile app, this research supports the inclusion of a remote-sensing intelligence module. The app can leverage publicly available satellite data (e.g., Sentinel-2, Landsat 8, MODIS) to provide users with regional soil-condition reports, salinity risk maps, drought indices, and crop suitability recommendations.

These maps enrich the user experience by offering context beyond immediate soil sampling results. However, the study also noted that remote-sensing-based nutrient estimation has limitations: spatial resolution may not capture small plots, cloud cover can reduce data availability, and chemical nutrients such as NPK cannot be directly quantified from satellite imagery.

This makes remote sensing a complementary tool rather than a replacement for in-field measurements. The app can integrate the two approaches—using smartphone colorimetry or sensors for precise point measurements and satellite data for broader insights. For example, the app could warn users if they are in a high-salinity region or recommend irrigation adjustments based on moisture indices derived from remote sensing. Abdulraheem et al. also highlighted the potential of combining remote sensing with machine learning for improved predictive models, enabling the app to provide forecasts for soil moisture trends or nutrient depletion over time.

The study guides the app's architecture by emphasizing multi-source data integration and the importance of delivering actionable, user-friendly insights rather than raw spectral data. Ultimately, incorporating remote sensing expands the app's capabilities, making it a comprehensive soil health advisory tool rather than merely a point-testing platform.

Zhai et al. (2024) examined machine-learning-driven classification techniques used for interpreting outputs from arrays of ion-selective electrodes (ISEs). Their review explained how combining multiple electrodes in a sensor array provides richer information than single electrodes, but interpreting that complex data requires sophisticated classification algorithms. They evaluated machine learning methods including support vector machines (SVM), random forest, principal component analysis (PCA), artificial neural networks (ANN), partial least squares regression (PLSR), and deep learning models. These models help differentiate ion signals, correct for cross-interference, and enhance the precision of nutrient estimation.

The implications for a soil-testing app are substantial. Soil is a chemically complex matrix containing numerous ions that can interfere with one another. For example, nitrate electrodes may respond slightly to chloride or bicarbonate ions, causing measurement errors. Zhai et al.'s findings reveal that integrating ML-based correction algorithms significantly improves the reliability of sensor-based soil testing. A soil-testing app can incorporate cloud-based or on-device machine learning models trained on large calibration datasets to interpret ISE-array outputs more accurately. This ensures that field measurements remain consistent across diverse soil types.

The review also emphasized that machine learning can help maintain accuracy even when sensors degrade over time, as algorithms can identify drift patterns and compensate accordingly. For the app, this supports the development of features such as automated drift detection, predictive calibration reminders, and error-correction filters. Additionally, Zhai et al. noted that multi-sensor arrays paired with ML improve multi-nutrient testing efficiency—something highly beneficial for soil diagnosis where nitrogen, potassium, pH, and salinity must often be tested simultaneously.

Their findings guide the architecture of a more advanced version of the soil-testing app. Instead of relying solely on raw sensor output, the app can integrate ML to analyze sensor-array data, generating more accurate nutrient recommendations and reducing user-dependent variability. This makes the app suitable not only for farmers but also for agricultural researchers who require high-quality data.

da Silva et al. (2025) developed an advanced AI-assisted soil pH classification framework that uses smartphone imaging of paper-based pH indicators to provide highly accurate field measurements. Their work addressed one of the most persistent challenges in mobile soil testing: the variability introduced by uncontrolled lighting, inconsistent camera sensors, and differences in user technique. By combining low-cost paper indicator strips with machine learning models, the study demonstrated that smartphones can deliver results comparable to laboratory-based pH meters, even under non-ideal field conditions. The researchers captured images of colorimetric pH papers under various lighting conditions—sunlight, shade, indoor light, diffused light, and mixed environments—to intentionally test the robustness of machine learning algorithms. This comprehensive dataset laid the foundation for building a classification model capable of handling real-world variability.

A key innovation of the study was its use of convolutional neural networks (CNNs), which were trained to recognize subtle color variations corresponding to different pH levels. Traditional colorimetric analysis using raw RGB values often struggles to maintain accuracy because colors shift depending on shadows, phone camera sensors, or exposure differences. The CNN-based approach overcomes this by learning deeper, more abstract color–pattern relationships, enabling it to decode pH-level information even from noisy or imperfect images. The researchers also compared CNN performance with classical supervised ML models such as Support Vector Machines (SVM), Random Forests, and K-Nearest Neighbors (KNN). While traditional models performed moderately well, CNNs consistently delivered the highest accuracy, underscoring the importance of computer vision techniques in smartphone-based soil analysis.

The relevance of this work to a soil-testing app is substantial. The study demonstrates that machine learning can effectively correct for nearly all the major factors that degrade mobile testing accuracy—lighting inconsistency, variations in phone camera hardware, differences in user angle and distance, and non-uniform backgrounds. For a soil-testing app built around smartphone colorimetry, these challenges are unavoidable. By implementing a similar ML pipeline, the app can classify not just pH but a broad range of nutrient-based colorimetric reactions such as nitrate, phosphorus, potassium, iron, zinc, and salinity indicators. The system can also employ preprocessing steps highlighted in the study, including color normalization, histogram correction, and reference patch recognition, to further enhance reliability.

The researchers emphasized several practical design recommendations that align closely with mobile application development. For instance, they noted the importance of guiding users to place the pH strip on a uniform background to minimize the influence of environmental colors. A soil-testing app can implement this by integrating AR-based alignment frames, step-by-step capture instructions,

auto-exposure locking, and real-time validation that checks whether the lighting is adequate. da Silva et al. also highlighted the utility of including reference color patches within the imaging frame. These patches act as ground-truth calibration points that allow the model to adjust for variations in lighting and camera sensors. The app can adopt this approach by encouraging users to purchase low-cost testing kits that include standardized reference strips or by digitally embedding color guides that the app uses for calibration.

One of the most impactful findings of the study was the feasibility of on-device ML inference, meaning the classification model can run directly on the smartphone without requiring cloud connectivity. This capability is crucial for rural agricultural users who may lack stable internet access. The authors demonstrated that optimized CNN models could run efficiently even on mid-range smartphones, delivering results instantly and enabling real-time field use. For a soil-testing app intended for farmers, this ensures accessibility, reliability, and speed regardless of network conditions. It also improves data privacy because sensitive agricultural data can remain entirely on the device.

The study ultimately provides powerful validation for integrating machine learning–enhanced computer vision into soil testing applications. It shows that smartphone imaging combined with AI not only compensates for field variability but can achieve near-laboratory accuracy in measuring essential soil parameters. By adopting the methodologies described by da Silva et al., a soil-testing app can significantly enhance its accuracy, user trust, and overall practicality. The research helps establish the scientific foundation for turning any smartphone into a portable soil-testing lab, making advanced agricultural diagnostics affordable and accessible for smallholder farmers worldwide.

Gunasekaran (2025) introduced a comprehensive real-time soil fertility prediction prototype that demonstrated how sensor fusion and machine learning (ML) can significantly enhance the accuracy and usability of soil analysis systems. The study integrated a multi-sensor hardware module capable of capturing a variety of soil parameters in real time — including pH, soil moisture, electrical conductivity (EC), temperature, nitrate concentration, ammonium content, and sometimes soil salinity proxies. These sensors, when used together, generated a rich dataset that allowed the machine learning models to detect soil fertility trends with far greater precision compared to single-parameter systems. A critical contribution of the study was showing that soil properties often interact in non-linear ways; therefore, single-measurement evaluations fail to capture the real nutrient dynamics. By feeding simultaneous multi-sensor data into ML algorithms, the system was able to model these relationships more accurately and generate reliable fertility indices.

The research implemented two major ML approaches: Random Forest (RF) and Artificial Neural Networks (ANN). The random forest model leveraged its ensemble learning nature to handle noisy sensor data and deliver stable predictions, while the neural network model excelled in detecting complex patterns across parameters. Both models were trained using datasets where each sensor reading was paired with laboratory-tested ground-truth values, ensuring scientific robustness. Gunasekaran's findings indicated that sensor fusion combined with ML reduced prediction errors significantly, overcoming challenges posed by soil heterogeneity, sensor drift, and environmental fluctuations. This advancement is important

because real-world soil conditions are rarely consistent, and the ability to adapt to variability directly improves the reliability of digital agriculture systems.

The study is particularly valuable for building advanced functionalities in a soil-testing mobile application. Gunasekaran's architecture outlines how a mobile app can accept diverse input sources — such as sensor-module readings, user-entered field observations, weather data, or smartphone-based measurements — and transmit them to a cloud-based or on-device ML engine. The ML engine then predicts fertility status, identifies nutrient deficiencies, classifies soil types, and even recommends fertilizer application rates. Such an approach transforms the app from a basic diagnostic tool into a smart advisory system. It can offer instant, tailored guidance such as recommended doses of nitrogen, phosphorus, or potassium, alerts about impending soil-health decline, or crop-specific soil suitability insights.

The research also highlighted several practical challenges that must be considered when developing real-world ML-driven agriculture applications. One major challenge is the requirement of high-quality, region-specific calibration datasets. Because soils differ widely across geographic regions in texture, mineral content, organic matter levels, and climatic influences, ML models must be trained on data that reflects these variations. Without regionally-inclusive datasets, prediction accuracy declines. This insight suggests that the soil-testing app should incorporate mechanisms for continuous data collection from users — for example, anonymized sensor readings paired with occasional lab samples. Over time, this would allow the ML backend to become increasingly accurate for different soil types and agro-climatic zones.

Another challenge identified in the study is data noise — especially from low-cost sensors that may drift over time or behave inconsistently across soil textures. Gunasekaran proposed periodic calibration routines, filtering algorithms, and self-correction mechanisms using ML, all of which can be embedded inside the app. These strategies enable long-term reliability without requiring farmers to replace sensors frequently.

Gunasekaran also emphasized the importance of real-time decision support, which aligns closely with modern precision agriculture. Integrating such capabilities into the soil-testing app means that farmers could receive dynamic updates whenever soil conditions cross critical thresholds. The app could provide time-series visualizations of key parameters, weekly soil-health summaries, seasonal trends, and predictive alerts for nutrient depletion. These tools empower farmers to take immediate action, optimizing fertilizer use, improving crop yields, and reducing environmental impact. For technologically advanced users, the app could sync with Bluetooth-enabled sensor nodes in the field, supporting continuous soil monitoring.

Ultimately, Gunasekaran (2025) demonstrates the path toward building an AI-powered soil health ecosystem**.** The study validates that multi-sensor fusion combined with machine learning produces superior fertility predictions compared to conventional methods. It shows that when these components are integrated with mobile platforms, real-time soil intelligence becomes accessible even to smallholder farmers. The soil-testing app can leverage these insights to build a

scalable, adaptive, and scientifically reliable decision-support system that evolves with every new data point. This research therefore plays a foundational role in shaping the AI and sensor-integration architecture of the proposed soil-testing application.

Sobhy et al. (2025) presented a futuristic, multi-layered review of soil nutrient monitoring systems categorized into five major approaches: traditional laboratory testing, remote sensing, IoT/smart systems, in situ sensors, and AI-driven prediction models. Their study highlighted the increasing synergy between hardware sensors, cloud platforms, and machine learning algorithms in modern precision agriculture. The authors emphasized that the future of soil fertility management lies in integrating multiple data sources—sensor readings, satellite data, weather information, historical datasets, and machine learning—to produce actionable, real-time recommendations.

For a soil-testing app, this research is foundational. It shows that the most effective digital soil-management system is not one that relies on a single method but one that unifies several complementary technologies. The app can adopt this hybrid architecture: smartphone-based nutrient testing for low-cost accessibility, optional electrochemical sensors for high-precision measurements, remote sensing for contextual insights, and AI models for interpreting and predicting soil conditions. Sobhy et al. stressed the importance of cloud-based analytics for model training, but also acknowledged the value of edge computing for real-time inference—something modern smartphones can handle.

The study also pointed out major barriers such as sensor maintenance, inconsistent farmer data entry, limited rural internet connectivity, and the need for localized calibration models. The app can address these issues through offline mode capabilities, built-in educational modules, and region-specific AI models. By combining the insights from this research, the app can evolve into a comprehensive soil-intelligence ecosystem supporting nutrient diagnosis, crop recommendations, fertilizer planning, and continuous monitoring.

Sobhy et al.'s work ultimately validates that building a soil-testing app based on the integration of multiple technologies is not only feasible but aligned with global trends in agricultural digitization. Their framework directly shapes the architecture and long-term roadmap of the project.

CHAPTER 3: REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

Agriculture remains the backbone of rural economies, especially in developing nations, where a large portion of the population depends on farming as their primary livelihood. Despite its importance, crop yields remain highly inconsistent due to poor soil management and limited access to accurate soil diagnostics. Farmers often rely on traditional, experience-based methods to judge soil quality—looking at texture, color, or past crop performance—which provides only a superficial understanding of soil health. In reality, soil fertility is governed by dynamic and complex chemical, physical, and biological interactions involving nutrient availability, pH balance, organic matter composition, moisture levels, salinity, and microbial activity. Without scientific soil analysis, farmers frequently over-apply or under-apply fertilizers, resulting in nutrient imbalances, reduced yields, soil degradation, increased costs, and environmental contamination due to nutrient runoff. The core problem, therefore, is the absence of an accessible, affordable, real-time soil testing and advisory system tailored for common users, especially small and marginal farmers.

Traditional soil testing laboratories exist, but they are often slow, expensive, geographically inaccessible, and require technical support that rural farmers cannot easily access. Reports from laboratories typically take several days or even weeks to return, leading to delays in agricultural decision-making. Moreover, farmers often struggle to interpret lab reports, which involve complex chemical indicators and technical fertilizer recommendations. These challenges create a significant gap between scientific soil analysis and practical field use. Additionally, many farmers are unaware of soil health importance or do not test soil regularly because of logistical barriers. Existing digital tools and apps offer limited functionality—either they provide generic fertilizer recommendations without field-level diagnostics or require external sensors that are too costly for widespread adoption.

Thus, the broader problem is two-fold:

(1) a lack of accessible soil testing solutions, and

(2) a lack of personalized, data-driven recommendations that ordinary users can easily understand.

A modern solution requires combining smartphone-based soil testing, sensor integration, machine learning, and scientific agricultural knowledge into a single, unified system. The proposed Soil Testing Mobile Application aims to fill this gap by enabling users to determine soil properties using smartphone cameras, optional low-cost sensors, remote data sources, and intelligent analysis techniques. The app will guide the user step-by-step to obtain soil readings, interpret results, and receive fertilizer recommendations tailored to crop type, soil category, and environmental conditions.

To systematically address this challenge, the overall problem is divided into the following sub-problems:

- **Lack of Real-Time Soil Nutrient Evaluation**
  Farmers have no simple method to instantly measure nitrogen (N), phosphorus (P), potassium (K), pH, or micronutrients at their location. Laboratory-based testing takes time and cannot support rapid decision-making, especially during sowing periods when decisions must be made within hours or days. A robust digital system must perform quick computations using smartphone colorimetry, sensor inputs, or user-provided data.
- **Variability in Soil Properties Across Fields**
  Soil properties vary not just across regions but also within a single farm. One portion of the field may be nutrient-rich, while another may be nutrient-deficient. Farmers often treat the entire field uniformly due to a lack of localized diagnostics. The challenge is to build a system capable of handling location-based readings and maintaining a soil-history database that tracks changes over time for each plot entered by the user.
- **Difficulty in Interpreting Technical Soil Reports**
  Even when soil reports are available, smallholder farmers struggle to understand the scientific terminology like "available phosphorus," "CEC," "organic carbon percentage," or "base saturation." A soil-testing app must convert complex soil parameters into simple insights: "Your soil is low in nitrogen, apply 40 kg urea per acre," or "Your pH is too acidic; apply lime." The system must convert scientific data into understandable, actionable recommendations.
- **Limited Access to Expensive Sensors and Laboratory Tools**
  High-end sensors, optical devices, and spectroscopy-based soil analyzers are too costly for widespread adoption. Therefore, the challenge is to build a solution that works with zero external hardware (only a smartphone) while also allowing optional low-cost sensor integration (pH sensor, EC meter, moisture sensor, NPK reader) for advanced users. The system must support both categories without compromising accuracy.
- **Environmental Factors Affecting Smartphone Readings**
  Smartphone-based colorimetric readings can be affected by lighting, shadows, camera type, background interference, or improper angles. This problem must be addressed through guided user instructions, calibration routines, reference color patches, and possibly AI/ML-based correction algorithms that adjust for variations in lighting and device quality.
- **Need for Location-Based Soil and Weather Integration**
  Crop recommendations depend heavily on local climate, rainfall, soil type, and cropping patterns. Without integrating weather APIs, soil maps, and regional crop databases, recommendations remain generic and imprecise. The problem is to combine local environmental data with soil-test results to produce accurate advisory outputs.
- **Absence of Historical Soil Tracking**
  Soil health is not a one-time measurement—it must be monitored over months or years. Farmers rarely maintain long-term soil health records. The problem is to design a digital platform that stores, visualizes, and compares soil test results over time, enabling users to track improvements or deterioration.
- **Need for Machine-Learning-Driven Fertilizer Recommendations**

Traditional fertilizer recommendations follow fixed formula tables that do not consider variations in soil composition, crop variety, climate, and user behavior. ML models can analyze historical readings, environmental conditions, and real-world outcomes to generate more personalized recommendations. The challenge lies in acquiring training data, validating model accuracy, and ensuring reliable predictions.

- **Ensuring Offline Functionality for Rural Users**
  Many rural farmers have poor internet access. A soil-testing system that depends solely on cloud processing becomes unusable. The problem is to design the app so that core features (basic testing, pH classification, simple recommendations) work offline, while advanced features sync when the user regains connectivity.
- **User-Friendly Interface and Workflow**
  Most rural users are not tech-savvy. The challenge is to create a simple, intuitive interface where users can easily follow steps to test soil, capture images, read results, and understand recommendations without confusion. The system must minimize technical jargon and maximize usability.

3.2 Requirements Specification

The Requirements Specification defines *what* the proposed Soil Testing and Analysis System must do, independent of the technology that will be used to implement it. This section identifies the needs of farmers, agricultural officers, researchers, and system administrators, and highlights the limitations of the current manual soil testing process. The goal is to establish a clear functional and non-functional blueprint for the system.

3.2.1 Overview

The Soil Testing System aims to digitalize and automate the process of soil data collection, nutrient analysis, interpretation of results, fertilizer recommendations, and report generation. The system must store soil profile information, allow users to submit samples, process laboratory values, and generate advisories tailored to crop type and soil condition. It should improve accuracy, remove delays in traditional testing, and ensure easy accessibility of reports.

I. Functional Requirements

1. User Management Requirements

1. The system must allow different types of users such as farmers, lab technicians, administrators, and agricultural officers to log in.
2. Each user must have role-based access:
   - Farmers: submit soil data, view reports, check recommendations
   - Lab Technicians: input test results
   - Admin: manage users, configurations, system logs
3. New users must be able to register using phone number, email, or ID.

2. Soil Sample Collection Requirements

1. The system must allow users to enter soil sample details including:
   - Location (village, district, GPS)
   - Crop type
   - Soil texture
   - Irrigation type

2. The system must generate a Unique Sample ID for every submitted soil sample.
3. Users should be able to track their sample status (Submitted → Under Testing → Completed).

3. Laboratory Testing Requirements

1. Lab technicians must be able to enter soil parameters such as:
    - pH
    - EC (Electrical Conductivity)
    - Organic Carbon
    - Nitrogen (N)
    - Phosphorus (P)
    - Potassium (K)
    - Micronutrients (Zn, Fe, Mn, Cu, etc.)
2. The system must validate whether entered values fall within acceptable scientific ranges.
3. The system must store all laboratory readings in the database for future comparison and historical analysis.

4. Soil Analysis and Recommendation Engine

1. The system must automatically analyze soil nutrient levels.
2. It should categorize soil as:
    - Low, Medium, High for N, P, K
    - Normal/Alkaline/Acidic for pH
3. Based on analysis, the system must generate specific recommendations:
    - Fertilizer dose
    - Soil amendments (e.g., lime, gypsum)
    - Suitable crops for that soil
    - Water management suggestions
4. The recommendation engine must follow government agricultural guidelines (ICAR/Soil Mission).

5. Report Generation Requirements

1. The system must generate a detailed soil test report containing:
    - Farmer details
    - Sample details
    - Nutrient values
    - Interpretation
    - Recommendations
2. Reports should be generated in PDF format and downloadable.
3. Users must be able to share or print the report.
4. A color-coded nutrient bar/pie chart should be included for easy understanding.

6. Data Storage and Retrieval Requirements

1. All soil samples must be stored with timestamp and location details.
2. Farmers must be able to view:
    - Previous soil reports
    - Changes in nutrient levels over time

3. Admin must have access to all data for audit and analytics.

7. Notification Requirements

1. The system should send SMS/Email/App notifications when:
    - Sample is received
    - Testing is completed
    - Report is ready
2. Reminders for seasonal soil testing should be supported.

II. Non-Functional Requirements

1. Performance Requirements

1. Soil report generation should not take more than 3 seconds.
2. The system should support simultaneous logins (minimum 500 users at a time).
3. Database queries must execute efficiently to handle large datasets.

2. Reliability Requirements

1. The system should ensure zero data loss during report generation.
2. It must automatically back up soil testing data daily.
3. In case of a crash, the system should recover latest stored data.

3. Usability Requirements

1. The application must be easy to use for farmers with basic digital skills.
2. The interface should support multilingual options (English, Hindi, Marathi, etc.).
3. Icons, tooltips, and simplified terms must be used for better understanding.

4. Security Requirements

1. User authentication must be enforced through OTP or secure login.
2. Sensitive agricultural data must be encrypted.
3. Only authorized users must be able to edit soil testing results.
4. Admin must be able to track user activity logs.

5. Scalability Requirements

1. The system must allow future expansion:
    - more districts
    - additional soil parameters
    - new modules (weather integration, AI prediction)
2. The system should support cloud-based scaling.

6. Maintainability Requirements

1. Code should be modular for easy upgrades.
2. Database structure must support adding new nutrient fields without redesigning the entire system.

III. Existing System Analysis

The current manual soil testing system suffers from multiple issues:

1. Long delays (reports take 20–30 days).

2. Manual data entry mistakes lead to inaccuracies.
3. Difficulty tracking samples, resulting in lost or misplaced samples.
4. No centralized database, making historical analysis impossible.
5. Reports are handwritten, unprofessional, and hard to interpret.
6. Farmers do not receive timely notifications, causing improper fertilizer usage.
7. No advisory system, so farmers struggle to understand what actions to take.

The proposed system eliminates these issues by digitalizing the entire workflow.

3.3 Planning & Scheduling

3.3.1 Project Overview and Assumptions

This project implements a Soil Testing Web/Mobile Application with three major subsystems: (1) mobile-app + UI/UX, (2) backend services + database + cloud, and (3) sensors / ML modules (optional hardware integration). The team size assumed for schedule planning: 1 Project Manager (PM), 2 Backend Developers, 2 Frontend/Mobile Developers, 1 ML Engineer / Data Scientist, 1 Sensor/Hardware Engineer, 1 QA/Test Engineer, and 1 DevOps/Cloud Engineer. Work calendar: 5 working days/week. All durations below are in calendar working days.

1. **Requirement Collection**
   - Gather functional and non-functional requirements from farmers, agronomists, and domain experts.
   - Identify user needs such as soil testing, nutrient prediction, and fertilizer recommendation.
2. **Feasibility Study**
   - Assess technical feasibility (smartphone colorimetry, ML models, sensor integration).
   - Evaluate economic feasibility for small-scale farmers.
   - Study operational feasibility for rural regions with low connectivity.
3. **Scope Definition**
   - Define boundaries of the soil-testing system.
   - Specify what features are included (e.g., soil analysis, ML recommendations) and what is excluded.
4. **Work Breakdown Structure (WBS) Creation**
   - Break the project into modules: UI, backend, database, ML model, hardware integration, testing.
5. **Task Sequencing**
   - Determine logical order (requirements → design → development → testing → deployment).
   - Identify dependencies between tasks.
6. **Time Estimation**
   - Estimate minimum, maximum, and most realistic time for each task.
   - Use expert judgment and PERT for time calculation.
7. **Resource Planning**
   - Allocate developers, testers, designers, and ML engineers.
   - Assign hardware devices, soil sensors, and test kits.
8. **Technology Selection**
   - Finalize frameworks, programming languages, cloud services, database, and ML tools.

9. **Prototype Planning**
    - Plan initial UI mockups and functional prototypes.
    - Validate requirements through early demos.
10. **Quality Planning**
    - Define performance criteria, accuracy benchmarks, and usability standards.
    - Plan for regular testing cycles.
11. **Communication Planning**
    - Set up weekly meetings, progress reporting formats, and documentation protocols.
12. **Monitoring & Control Setup**
    - Create milestones and checkpoints for every phase.
    - Plan bug reviews, code audits, and sprint retrospectives.
13. **Risk Planning**
    - Prioritize risks and prepare mitigation strategies.
    - Assign responsibility to specific team members.
14. **Documentation Planning**
    - Define how each stage will be documented (requirements file, design documents, test reports).
15. **Deployment & Pilot Testing Plan**
    - Prepare rollout strategy, user pilot testing, feedback collection, and improvements roadmap.

3.3.2 Risk Analysis

1. **Requirement-Related Risks**
    - Incomplete or unclear requirements may cause rework.
    - Misunderstanding user needs may lead to feature gaps.
2. **Data Availability Risks**
    - Lack of sufficient soil datasets for ML training.
    - Difficulty in obtaining accurate lab-calibrated soil samples.
3. **Technical Risks**
    - ML model may produce inaccurate predictions in diverse soil conditions.
    - Colorimetric readings may fail under poor lighting.
    - Backend or database may experience performance issues.
4. **Hardware Integration Risks**
    - Sensors may malfunction or require frequent calibration.
    - Compatibility issues with microcontrollers or smartphones.
    - Hardware components may face delays or shortages.
5. **User Environment Risks**
    - Rural users may have limited internet connectivity.
    - Inconsistent lighting conditions affecting smartphone image capture.
    - Variability in soil types affecting model accuracy.
6. **Operational & Deployment Risks**
    - Application may fail during large-scale usage or peak load.
    - Cloud server downtime may affect real-time access.
7. **Team & Resource Risks**
    - Key team members may become unavailable.
    - Skill gaps in ML, sensor integration, or mobile development.
8. **Schedule Risks**

- Delays in design, data collection, or hardware testing may affect timelines.
- Overlapping tasks may cause bottlenecks.

9. **Security & Data Privacy Risks**
    - Risk of unauthorized access to user data or cloud server.
    - Weak authentication mechanisms.
10. **Financial Risks**
    - Unexpected hardware or cloud costs exceeding budget.
    - Cost of replacing faulty sensors or modules.
11. **User Adoption Risks**
    - Farmers may not trust the app initially.
    - Resistance to new technology in rural areas.
12. **Testing Risks**
    - Limited real-world pilot testing may reduce reliability.
    - Bugs in mobile UI or backend logic may go unnoticed.
13. **Maintenance Risks**
    - Continuous model retraining required due to soil variability.
    - Sensor wear-and-tear increasing long-term maintenance costs.

3.3.3 Gantt Chart

- Gantt Chart for Soil Testing App Development

Task / Month	**Nov**	**Dec**	**Jan**	**Feb**
Requirement Analysis				
System & UI Design				
Database + ML Planning				
App Development (Front + Back)				
ML Model Development				
Integration of Modules				
Testing & Bug Fixing				
Documentation & Deployment				

i. Gantt Chart

- PERT Chart

```
                    START
                      |
                      v
            [1] Requirement Analysis
                      |
                      v
              [2] System Design
                      |
                      v
        +-------------------------------------+
        |    Parallel Activities              |
        +-------------------------------------+
        |                                     |
        v                                     v
[3] UI/UX Design        [4] Backend Design
        |                                     |
        +-------------------------------+
                      |
                      v
            [5] Implementation
              - Frontend Dev
              - Backend Dev
             - ML Integration
                      |
                      v
              [6] Testing
             - Unit Testing
           - Integration Testing
             - Field Testing
                      |
                      v
            [7] Deployment
                      |
                      v
        [8] Documentation & Review
                      |
                      v

                    END
```

ii.PERT CHART

3.4 Software and Hardware Requirements

The Soil Testing Application requires a combination of software and hardware components to support development, testing, deployment, and operational use in real-world environments.

- **Software Requirements**
  - Development Tools

- Programming Languages: Python (ML), JavaScript/TypeScript, Dart (Flutter), Java/Kotlin (Android), Swift (iOS optional).
- Frameworks:
  - Frontend: React Native / Flutter
  - Backend: Node.js / Express / Django REST / Flask API
  - ML: TensorFlow, Scikit-Learn, Pandas, NumPy
- Database Systems: MySQL / PostgreSQL / MongoDB
- Version Control: Git, GitHub/GitLab
- IDE/Editors: VS Code, Android Studio, PyCharm
- API Testing Tools: Postman, Swagger UI
- Containerization: Docker (optional), Kubernetes for scaling
- Cloud Hosting: AWS / Google Cloud / Azure / Firebase
- Project Management Tools: Jira, Trello, Asana

- Testing & Deployment Software
  - Device emulators (Android AVD / iOS Simulator)
  - Selenium / Appium for automated testing
  - CI/CD tools such as GitHub Actions or Jenkins
  - Cloud monitoring dashboards (e.g., CloudWatch)

➢ Hardware Requirements

- Development Hardware
  - Laptops/desktops (minimum 8 GB RAM, recommended 16 GB+)
  - Smartphones for testing (Android 8+ and iOS 13+)
  - Local server or cloud-instance access
- Field & Sensor Hardware
  - Soil moisture sensor
  - pH sensor
  - Temperature sensor
  - NPK soil nutrient sensor (optional but beneficial)
  - Microcontroller (Arduino, ESP32, Raspberry Pi)
  - Bluetooth or Wi-Fi module for communication
  - Power supply or portable battery pack
  - Calibration kits for soil sensors
  - GPS module (optional) for location-based soil testing
- Networking & Supporting Devices
  - Internet router / hotspot for data sync
  - QR code scanner or smartphone camera
  - Storage devices for backup (external HDD/SSD)

CHAPTER 4: SYSTEM DESIGN

System design is a crucial phase of the Software Development Life Cycle (SDLC) that transforms system requirements into a structured, logical, and well-defined blueprint for implementation. It acts as a bridge between requirement analysis and actual coding, ensuring that the system is built in a systematic, efficient, and maintainable manner. This chapter presents the complete system design of the Soil Testing Web Application, describing its architecture, functional behavior, module organization, database structure, and procedural flow. The objective of this chapter is to provide a comprehensive understanding of how the different components of the system interact and operate together to deliver accurate soil analysis and meaningful agricultural recommendations.

The Soil Testing Web Application is designed with the primary goal of simplifying soil evaluation for farmers, gardeners, and agricultural learners. The system translates complex soil parameters into easy-to-understand insights, enabling users to make informed decisions regarding crop selection, fertilizer application, irrigation scheduling, and long-term soil health management. The design ensures that technical complexity is handled internally by the system while presenting a simple and intuitive interface to the end user.

A key focus of the system design is usability. The application is developed to be user-friendly so that even users with minimal technical knowledge can operate it without difficulty. Clear input forms, meaningful labels, validation messages, and well-structured result displays are incorporated to enhance the user experience. By minimizing manual effort and avoiding unnecessary complexity, the system ensures wider adoption among rural farmers and home gardeners.

Another important aspect of the design is scalability. The architecture of the Soil Testing Web Application allows for future expansion without major changes to the existing system. New features such as sensor-based soil data input, mobile application integration, advanced analytics, or machine learning-based predictions can be added seamlessly. The modular structure ensures that enhancements in one part of the system do not negatively impact other components.

The system is also designed to be reliable and accurate, which is critical for agricultural decision-making. Soil analysis and recommendations are generated using standardized agricultural thresholds and validated logic to ensure consistency in results. Proper error handling mechanisms are included to manage invalid inputs, missing values, and system exceptions. This improves the overall robustness of the application and ensures dependable performance under different conditions.

From an architectural perspective, the Soil Testing Web Application follows a client–server model. The frontend is responsible for user interaction, data input, and result visualization, while the backend handles data processing, soil analysis logic, database operations, and integration with external services such as weather APIs. This separation of concerns improves maintainability, security, and performance.

In addition to usability, scalability, and reliability, the system design also emphasizes modularity and maintainability. The Soil Testing Web Application is divided into well-defined modules, each responsible for a specific functionality such as user management, soil data processing, recommendation generation, and data storage. This modular

approach simplifies development and testing, allows independent updates to individual components, and reduces the overall complexity of the system. If any module requires enhancement or correction, changes can be made without affecting the entire application.

The design further incorporates data consistency and integrity as a fundamental requirement. Soil parameters entered by users are validated at multiple levels to prevent incorrect or unrealistic values from entering the system. Database constraints and structured schemas ensure that stored data remains accurate, consistent, and logically connected. This is particularly important when maintaining historical soil records, as reliable data is essential for tracking soil health trends over time and supporting informed agricultural decisions.

The system also focuses on performance efficiency, ensuring that soil analysis and recommendation generation occur with minimal delay. Optimized algorithms and efficient database queries are used to process user inputs quickly, even when the number of users or stored records increases. This ensures a smooth user experience and prevents system lag, which is especially important in real-time decision-support applications.

From a security standpoint, the design includes controlled access mechanisms to protect user data and system resources. Authentication and session management ensure that only authorized users can access their soil test records and recommendations. Sensitive information such as login credentials and user data is handled securely, reinforcing trust in the system and encouraging consistent usage.

The Soil Testing Web Application is also designed with extensibility in mind. While the current system supports manual data entry and rule-based analysis, the architecture allows for future integration of smart sensors, Internet of Things (IoT) devices, and machine learning models. These enhancements can improve accuracy, automate data collection, and provide predictive insights, further strengthening the application's value in modern agriculture.

Overall, the system design ensures that the Soil Testing Web Application is not only technically sound but also aligned with real-world agricultural needs. By combining a user-centric interface, structured backend logic, secure data handling, and scalable architecture, the design lays a strong foundation for effective implementation. This well-planned design approach minimizes development risks, improves system quality, and ensures that the application can evolve alongside advancements in digital agriculture and soil science.

- ✓ Key Design Principles Followed

  The design of the Soil Testing Web Application is guided by several fundamental software engineering principles that ensure system quality, effectiveness, and long-term usability. These principles help in developing a reliable, scalable, and user-centric system capable of supporting real-world agricultural needs.

  - **Modularity** is a core principle followed in the system design. The entire application is divided into independent functional units such as user management, soil data input, soil analysis, recommendation generation, weather integration, and reporting. Each module is designed to perform a specific task and interact with other modules through well-defined interfaces. This modular

structure simplifies development, enables parallel work, improves code readability, and allows individual modules to be tested and enhanced independently without affecting the overall system.

- **Reusability** is incorporated by designing components and functions that can be used across multiple parts of the application. Common functionalities such as input validation, soil parameter evaluation, database access, and result formatting are implemented as reusable components. This approach reduces code duplication, improves consistency across the system, and minimizes development effort when extending or modifying system features.
- **Security** is an essential design consideration due to the sensitive nature of user data and soil records. The system employs controlled user authentication and authorization mechanisms to ensure that only registered users can access the application. Secure data handling practices are followed to protect soil test records, user credentials, and recommendation data from unauthorized access. Session management and restricted access further strengthen system security and user trust.
- **Maintainability** is achieved through structured coding practices, clear naming conventions, and a layered architectural approach. The separation of frontend, backend, and database layers allows developers to locate and modify system components easily. Proper documentation and clean code structure simplify debugging, testing, and future enhancements, making the system easier to maintain over time.
- **Efficiency** is ensured by optimizing data processing logic and database operations. The soil analysis algorithms are designed to execute quickly while maintaining accuracy. Efficient database queries and structured data storage reduce processing overhead, allowing the system to handle multiple requests smoothly. This ensures that users receive real-time analysis and recommendations without noticeable delays.
- **Scalability** is also considered as a key principle. The system architecture allows for future expansion such as the integration of IoT-based soil sensors, mobile applications, advanced analytics, and machine learning models. The modular and layered design ensures that new features can be added with minimal changes to existing components.
- **Usability** is emphasized throughout the design process to ensure accessibility for users with varying levels of technical knowledge. Simple navigation, clear instructions, user-friendly forms, and informative result displays improve user interaction and encourage adoption among farmers and gardeners.

Overall, the application of these design principles ensures that the Soil Testing Web Application is robust, adaptable, and capable of delivering accurate and reliable agricultural recommendations while supporting future technological advancements.

4.1 UML Diagrams

UML (Unified Modeling Language) modeling is used to understand the problem domain and represent the system functionality in a structured and standardized manner. It provides a visual and conceptual representation of how the system behaves, how its components are organized, and how data flows between different entities. UML modeling plays a vital role during the design phase as it helps transform abstract system requirements into a clear and organized system blueprint before actual implementation.

In the Soil Testing Web Application, UML models are used to describe interactions between users and the system, relationships among system components, and the overall operational flow. The conceptual design illustrates how soil data is collected, processed, analyzed, and stored, and how meaningful recommendations are generated and presented to users. These models help developers, analysts, and evaluators clearly understand the system structure and ensure that all functional requirements are addressed efficiently.

The system primarily involves a single external actor, the user, who interacts with the application to perform operations such as registration, login, soil data entry, soil analysis, viewing recommendations, and accessing historical records. Internally, the system manages various processes including soil parameter validation, analysis logic execution, recommendation generation, weather data integration, and database management. UML modeling ensures clarity, consistency, and completeness in system design while reducing ambiguity during development.

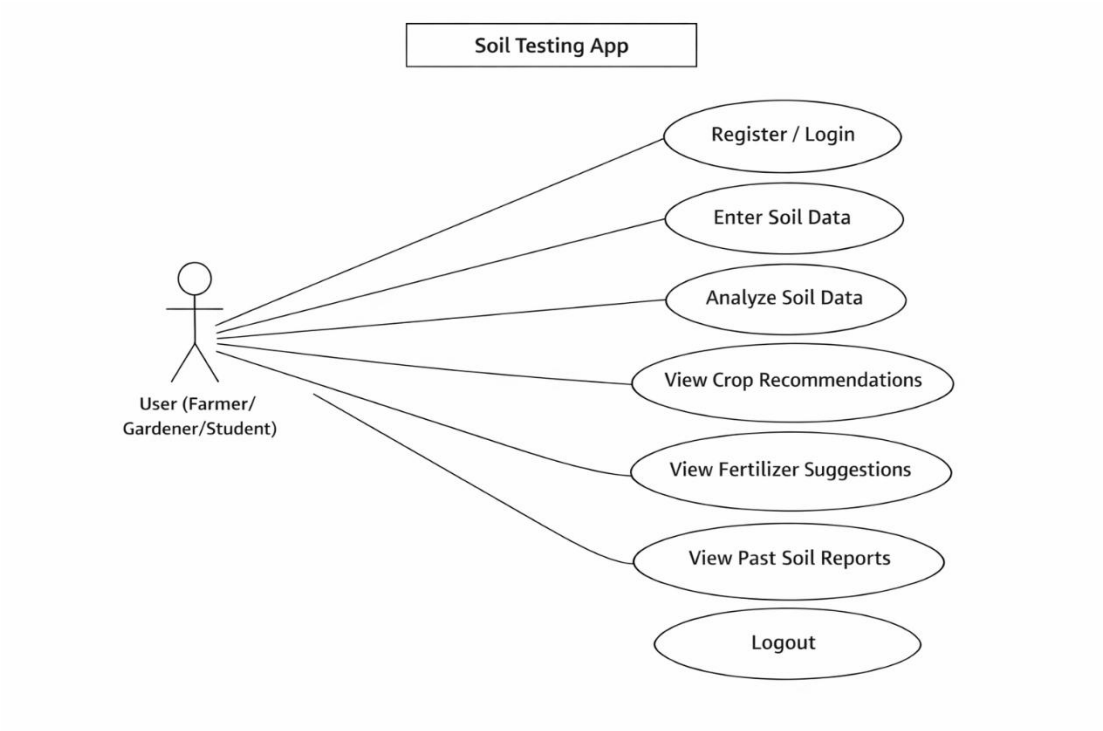
4.1.1 Use Case Diagram

The Use Case Diagram represents the functional requirements of the Soil Testing Web Application from the user's perspective. It identifies the different actions a user can perform while interacting with the system and highlights the system's behavior in response to those actions.

The Use Case Diagram represents the interaction between the user and the Soil Testing Web Application and highlights the main functionalities provided by the system.

- Actor (User – Farmer/Gardener):
  Represents the end user who interacts with the system to analyze soil and receive recommendations.
- System:
  Acts as the central component that processes user inputs and generates outputs such as soil analysis and crop recommendations.
- Login / Register:
  Allows the user to create an account or authenticate themselves to access system features securely.
- Enter Soil Data:
  Enables the user to input soil parameters such as pH, nitrogen, phosphorus, potassium, and moisture values.
- View Soil Analysis:
  Displays the analyzed soil condition, including nutrient sufficiency or deficiency.
- Get Crop Recommendation:
  Provides suitable crop suggestions and fertilizer advice based on soil analysis.
- View History:
  Allows the user to view previous soil test records and recommendations.
- Logout:
  Terminates the user session securely.

This diagram helps in understanding system boundaries and user-system interaction clearly.



iii. 4.1 Use Case Diagram

4.1.2. Class Diagram

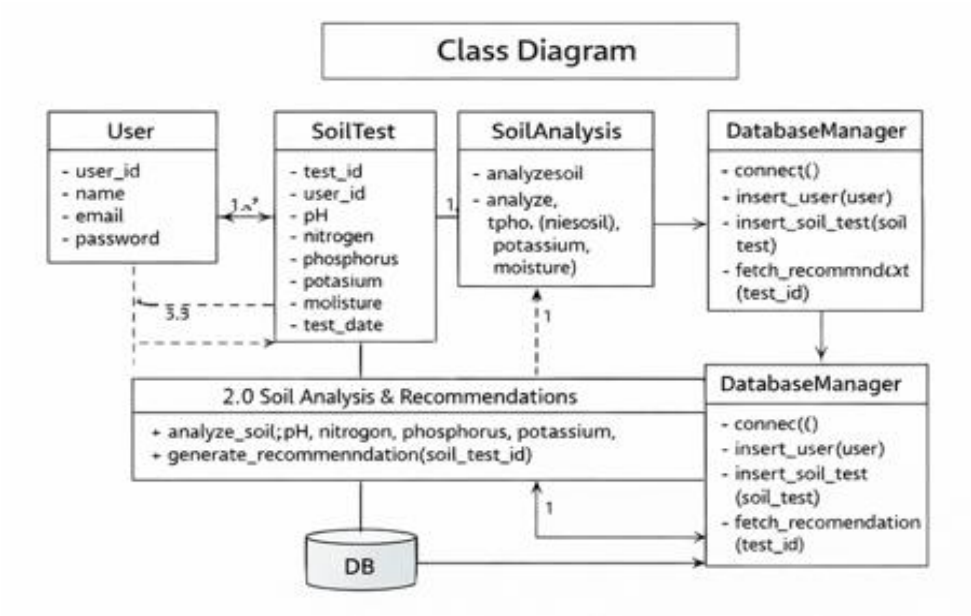
The Class Diagram describes the static structure of the system by illustrating the classes involved in the application, along with their attributes, methods, and the relationships between them. It represents the blueprint of the object-oriented structure of the Soil Testing Web Application. Unlike behavioral diagrams, which focus on system activities and interactions, the class diagram concentrates on how the system is structured internally. It defines how data is organized and how different components of the application are logically connected.

In this system, the primary classes include User, SoilTest, SoilAnalysis, and Recommendation. Each class is designed to handle a specific responsibility within the application. The *User* class manages user-related information such as registration and authentication. The SoilTest class stores soil parameter details entered by the user. The SoilAnalysis class contains the core processing logic that evaluates soil nutrient levels and determines deficiencies. Finally, the Recommendation class generates crop and fertilizer suggestions based on the analysis results. This separation of responsibilities follows the principle of modular and object-oriented design, which improves maintainability and scalability.

The relationships between these classes are clearly defined in the diagram. A one-to-many relationship exists between the User and SoilTest classes, as a single user can perform multiple soil tests. Additionally, a one-to-one relationship exists between SoilTest and *Recommendation*, since each soil test generates a specific recommendation. The SoilAnalysis class interacts with the SoilTest class to process input data and produce results. By defining these relationships and structures clearly, the Class Diagram ensures that the system design is organized, efficient, and easy to implement in code.

Main Classes:

1. User
   - Attributes:
     - user_id
     - name
     - email
     - password
   - Methods:
     - register()
     - login()
2. SoilTest
   - Attributes:
     - test_id
     - user_id
     - pH
     - nitrogen
     - phosphorus
     - potassium
     - moisture
     - test_date
3. SoilAnalysis
   - Methods:
     - analyze_soil()
     - check_nutrient_levels()
4. Recommendation
   - Attributes:
     - recommendation_id
     - soil_test_id
     - crop_suggestion
     - fertilizer_suggestion
   - Relationships:
     - One User can have many SoilTest records (1:N)
     - Each SoilTest generates one Recommendation (1:1)
     - SoilAnalysis processes SoilTest data



iv. 4.2 Class Diagram

4.1.3 Entity Relationship (ER) Diagram

The ER Diagram shows the database entities and their relationships. It provides a clear representation of how data is structured, stored, and interconnected within the system. This diagram is essential for designing a well-organized database that minimizes redundancy and ensures consistency. In the Soil Testing Web Application, the ER diagram defines the core entities involved in storing user information, soil test records, and generated recommendations.

The primary entities in the system are User, Soil_Test, and Recommendation. The *User* entity stores basic account information such as user_id, name, email, and password, where user_id acts as the Primary Key to uniquely identify each user. The Soil_Test entity stores detailed soil parameter values including pH, nitrogen, phosphorus, potassium, moisture, and test_date. The test_id serves as the Primary Key, while user_id acts as a Foreign Key linking each soil test to a specific user. The Recommendation entity contains recommendation_id as its Primary Key and test_id as a Foreign Key, along with crop_suggestion and fertilizer_suggestion, which store the output generated by the system.

The relationships between these entities are clearly defined to maintain referential integrity. A one-to-many (1:N) relationship exists between the *User* and Soil_Test entities, as a single user can perform multiple soil tests over time. Additionally, a one-to-one (1:1) relationship exists between Soil_Test and Recommendation, since each soil test produces a specific recommendation. These structured relationships ensure proper data linkage, eliminate duplication, and maintain consistency within the database. The ER diagram therefore serves as the foundation for implementing a reliable and normalized database structure.

Entities:

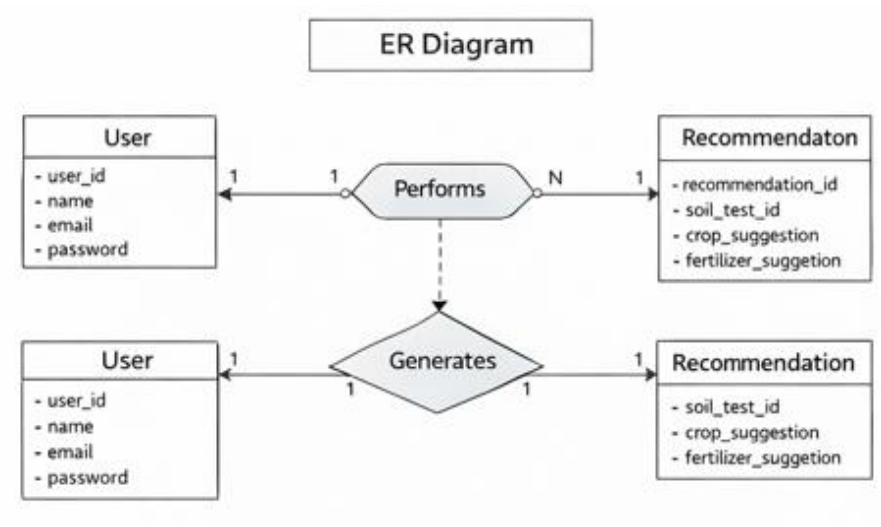
- **User**
  - user_id (Primary Key)
  - name

  - email
  - password

- **Soil_Test**
  - test_id (Primary Key)
  - user_id (Foreign Key)
  - pH
  - nitrogen
  - phosphorus
  - potassium
  - moisture
  - test_date

- **Recommendation**
  - recommendation_id (Primary Key)
  - test_id (Foreign Key)
  - crop_suggestion
  - fertilizer_suggestion

- **Relationships:**
  - User performs Soil_Test (1:N)
  - Soil_Test generates Recommendation (1:1)



v. 4.3 ER Diagram

4.1.4 System Flowchart

The System Flowchart describes the overall working process of the Soil Testing Web Application in a structured and sequential manner. It visually represents how control flows from one operation to another, starting from user interaction to final result generation. A flowchart is important because it simplifies complex system logic into understandable steps, making it easier for developers and evaluators to interpret the system's behavior.

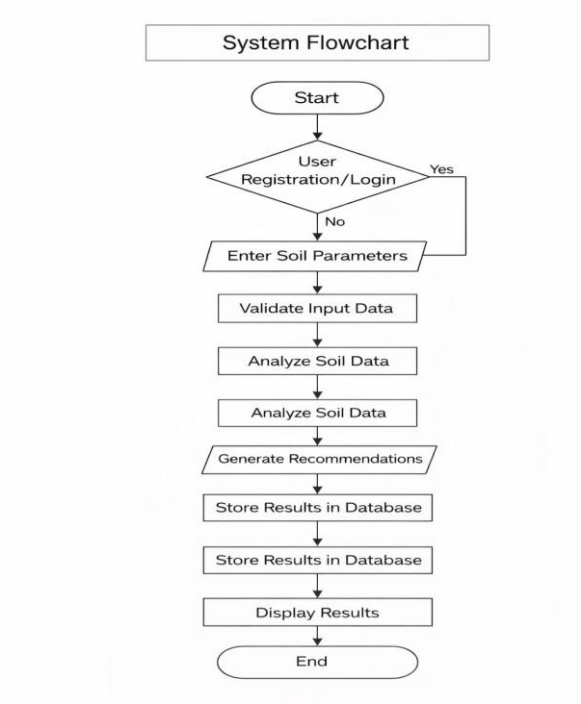
In this application, the flow begins when the user accesses the system and either registers or logs in. After successful authentication, the user is directed to the soil input interface where soil parameters such as pH, nitrogen, phosphorus, potassium, and

moisture are entered. The system then validates the input data to ensure correctness and prevent invalid values. Once validation is complete, the analysis module processes the data and generates crop and fertilizer recommendations. These results are stored in the database for future reference and then displayed to the user.

The flowchart ensures that every operation follows a logical sequence without skipping essential steps. It also helps in identifying decision points such as input validation and authentication checks. By clearly defining the system workflow, the flowchart improves understanding, debugging, and future enhancements of the application.

Flow Description

1) **Start**
   The system begins execution when the user opens the application.
2) **User Registration/Login**
   - If the user is new → Register.
   - If already registered → Login.
   - Credentials are verified.
3) **Enter Soil Parameters**
   The user inputs soil values such as pH, Nitrogen, Phosphorus, Potassium, and Moisture.
4) **Validate Input Data**
   - Check if fields are empty.
   - Ensure values are within acceptable ranges.
   - If invalid → Display error message.
   - If valid → Proceed to analysis.
5) **Analyze Soil Data**
   The system compares input values with predefined agricultural standards to determine soil condition and nutrient deficiencies.
6) **Generate Recommendations**
   Based on analysis, the system suggests suitable crops and fertilizers.
7) **Store Results in Database**
   Soil test details and recommendations are saved for future reference.
8) **Display Results**
   The final recommendations are shown to the user on the screen.
9) **End**
   The process completes successfully.



vi. 4.4 System Flowchart

4.2 Module Design

The system is divided into independent modules using the divide-and-conquer strategy. This approach breaks the overall system into smaller, manageable components, making development, testing, and maintenance easier. Each module is designed to perform a specific function and operates independently while interacting with other modules through well-defined interfaces. This modular structure enhances system scalability, improves code readability, and simplifies debugging and future upgrades.

The modular design ensures that changes in one module do not significantly affect other modules. For example, if improvements are required in the recommendation logic, modifications can be made in the Recommendation Module without altering the User Management or Database Modules. This separation of concerns follows software engineering best practices and supports structured system development.

1. User Management Module

The User Management Module is responsible for handling all user-related operations within the system. It manages user registration, login authentication, and session control. When a new user registers, the module collects required information such as name, email, and password. The password is securely stored in encrypted form to maintain data security.

During login, the module verifies user credentials by comparing the entered email and password with the stored database records. If the credentials are valid, access is granted; otherwise, an error message is displayed. The module also manages user sessions to ensure that only authenticated users can access soil testing features.

Additionally, this module ensures:

- Prevention of duplicate email registrations
- Proper validation of user input fields
- Secure logout functionality
- Protection against unauthorized system access

By managing authentication and authorization effectively, this module ensures system security and controlled access.

2. Soil Data Input Module

The Soil Data Input Module provides an interface for users to enter soil parameters. It includes input forms where users can enter values such as pH level, Nitrogen, Phosphorus, Potassium, and Moisture content. This module acts as a bridge between the user and the analysis engine.

One of the key responsibilities of this module is input validation. It ensures:

- No field is left empty
- Numeric values are entered where required
- Nutrient values are within acceptable agricultural ranges
- No negative or invalid values are submitted

If invalid data is entered, the system immediately displays appropriate error messages, preventing incorrect data from reaching the analysis stage. This improves system reliability and ensures accurate recommendations.

The module also formats the input data properly before sending it to the Soil Analysis Module. This structured handling of input data minimizes processing errors and enhances system stability.

3. Soil Analysis Module

The Soil Analysis Module is the core processing unit of the application. It performs the main computational tasks of evaluating soil parameters. This module compares user-provided soil values with predefined agricultural standards stored within the system.

The analysis process involves:

- Checking whether pH levels are acidic, neutral, or alkaline
- Identifying nutrient deficiencies (low nitrogen, phosphorus, potassium)
- Detecting excess nutrient levels
- Evaluating soil moisture conditions

Based on these comparisons, the module determines the overall soil condition. The logic used in this module is rule-based, meaning predefined conditions trigger specific outputs. For example:

- If Nitrogen is below the minimum threshold → Mark as Nitrogen Deficient
- If $pH < 6$ → Soil is acidic
- If $pH > 7.5$ → Soil is alkaline

The results generated by this module are forwarded to the Recommendation Module for further processing. Since this is the central logic component, it is designed carefully to ensure accurate and consistent results.

4. Recommendation Module

The Recommendation Module generates crop and fertilizer suggestions based on the analysis results. It uses rule-based logic to match soil conditions with suitable agricultural recommendations.

For example:

- If soil is nitrogen deficient → Recommend nitrogen-rich fertilizers
- If soil is acidic → Suggest lime treatment
- If pH and nutrients are balanced → Recommend crops suitable for balanced soil

The module ensures that recommendations are clear, practical, and easy to understand for users. It may also categorize crops based on soil suitability, helping farmers make informed decisions.

This module improves agricultural productivity by:

- Reducing guesswork in crop selection
- Encouraging proper fertilizer usage
- Promoting balanced soil nutrient management

The output generated here is stored in the database and displayed to the user through the interface.

5. Database Module

The Database Module is responsible for handling all data-related operations. It manages the storage, retrieval, update, and deletion (CRUD operations) of user records, soil test data, and recommendation results.

Its primary responsibilities include:

- Storing user registration details
- Saving soil test reports
- Storing generated recommendations
- Fetching previous reports when requested
- Maintaining referential integrity between tables

This module ensures persistent data storage, meaning that information remains saved even after the user logs out. It also ensures data consistency by enforcing primary keys, foreign keys, and constraints.

Additionally, the Database Module:

- Prevents duplicate records
- Ensures secure data transactions
- Maintains backup and recovery mechanisms

Efficient database management improves overall system performance and ensures reliability.

6. Integration of Modules

Although each module operates independently, they are integrated to function as a complete system. The workflow follows a structured sequence:

1. User logs in (User Management Module).
2. User enters soil data (Soil Data Input Module).
3. Data is analyzed (Soil Analysis Module).
4. Recommendations are generated (Recommendation Module).
5. Results are stored and retrieved (Database Module).

This integration ensures smooth communication between modules while maintaining independence.

Advantages of Modular Design

- Improves system maintainability
- Simplifies debugging and testing
- Enhances scalability
- Supports future feature additions
- Encourages code reusability
- Reduces system complexity

The modular design of the Soil Testing Web Application ensures structured development and efficient functionality. By dividing the system into logical components, the application becomes easier to manage, maintain, and expand. Each module performs a dedicated task, and together they create a fully functional and reliable system for soil analysis and recommendation generation.

4.3 Database Design

Database design defines how data is organized, stored, managed, and accessed within the system. In the Smart Soil Testing and Monitoring System, the database plays a crucial role in storing user information, soil test records, and related data required for analysis and retrieval.

A structured database is essential to ensure that soil data such as pH values, moisture levels, and nutrient content (nitrogen, phosphorus, potassium) are stored accurately and can be accessed efficiently whenever required. It also supports functionalities like user authentication, data history tracking, and statistical analysis.

The system uses a relational database such as MySQL to provide reliable data storage, fast query execution, and efficient data management. The database is designed to maintain data integrity, avoid redundancy, and support multiple operations such as insertion, retrieval, updating, and deletion of records.

Proper database design ensures that all modules of the system—such as user management, soil testing, statistics, and weather integration—work seamlessly together. It also improves system performance and allows future scalability, such as integration with cloud storage or large-scale agricultural data systems.

4.3.1 Schema Design

Schema design defines the structure of the database by specifying tables, fields, data types, and relationships between entities. In the Soil Testing Web Application, the database is designed to store user information, soil test records, and generated recommendations in a structured and organized manner. The schema ensures data consistency, eliminates redundancy, and maintains relationships using primary and foreign keys.

The system consists of three main tables: User, Soil_Test, and Recommendation. Each table plays a specific role in managing and storing system data.

1. User Table

The User table stores all registered user details. It is responsible for managing authentication and identifying system users uniquely.

Field Name	Type	Description
user_id	INT (PK)	Unique identifier
name	VARCHAR(100)	User name
email	VARCHAR(100)	Unique email
password	VARCHAR(100)	Encrypted password

a. 4.1 User Table

Explanation:

- user_id is the Primary Key of the table. It uniquely identifies each user in the system and prevents duplicate records.
- name stores the full name of the user for identification and personalization purposes.
- email is used as a login credential and is marked as unique to prevent duplicate registrations.
- password stores the encrypted version of the user's password to ensure security and protect sensitive information.

This table plays a crucial role in authentication and authorization processes. It establishes a one-to-many relationship with the Soil_Test table, as one user can perform multiple soil tests.

2. Soil_Test Table

The Soil_Test table stores detailed soil parameter values entered by users. It acts as the core data table for soil analysis.

Field Name	Type	Description
test_id	INT (PK)	Unique test ID
user_id	INT (FK)	Linked to User
pH	FLOAT	Soil acidity level
nitrogen	FLOAT	Nitrogen content
phosphorus	FLOAT	Phosphorus content
potassium	FLOAT	Potassium content

Field Name	Type	Description
moisture	FLOAT	Soil moisture
test_date	DATE	Date of testing

b 4.2 Soil Test

Explanation:

- test_id is the Primary Key and uniquely identifies each soil test record.
- user_id is a Foreign Key that references the User table, linking each soil test to a specific user.
- pH stores the acidity or alkalinity level of the soil.
- nitrogen, phosphorus, potassium represents the essential nutrient values required for plant growth.
- moisture indicates the water content of the soil.
- test_date records when the soil test was performed.

This table is central to the system because it stores the raw input data used for analysis. Each record in this table is processed by the Soil Analysis Module to generate recommendations.

3. Recommendation Table

The **Recommendation** table stores the output generated after soil analysis. It contains crop and fertilizer suggestions based on soil conditions.

Field Name	Type	Description
recommendation_id	INT (PK)	Unique ID
test_id	INT (FK)	Linked to Soil_Test
crop_suggestion	VARCHAR(200)	Suggested crop
fertilizer_suggestion	VARCHAR(200)	Suggested fertilizer

c 4.3 Recommendation Table

Explanation:

- recommendation_id is the Primary Key that uniquely identifies each recommendation entry.
- test_id is a Foreign Key that links the recommendation to a specific soil test record.
- crop_suggestion stores the recommended crop suitable for the analyzed soil.
- fertilizer_suggestion stores the recommended fertilizer to improve soil quality.

There is a one-to-one relationship between Soil_Test and Recommendation because each soil test generates exactly one recommendation record.

Overall Schema Structure

The schema is designed using relational database principles. The relationships between tables ensure:

- One User → Many Soil_Test records (1:N relationship)
- One Soil_Test → One Recommendation (1:1 relationship)

The use of primary and foreign keys maintains referential integrity and prevents orphan records. This structured schema design ensures efficient storage, quick retrieval, and secure management of soil testing data.

4.3.2 Data Integrity and Constraints

To maintain data accuracy, consistency, and reliability within the database, several integrity constraints are implemented. These constraints prevent invalid data entry and ensure proper relationships between tables.

Primary Key Constraint: Ensures that each record in a table is uniquely identifiable. It prevents duplicate entries by enforcing uniqueness in fields such as user_id, test_id, and recommendation_id.

Foreign Key Constraint: Maintains relationships between tables by ensuring that a foreign key value must exist in the referenced primary key table. This preserves referential integrity between User, Soil_Test, and Recommendation tables.

Unique Constraint (Email Field): Ensures that no two users can register with the same email address. This prevents duplicate accounts and supports secure authentication.

NOT NULL Constraint: Ensures that mandatory fields such as name, email, password, and soil parameters cannot be left empty. This guarantees that incomplete records are not stored in the database.

CHECK Constraint (pH Range): Restricts the pH value to remain between 0 and 14. This ensures scientifically valid soil acidity values are stored.

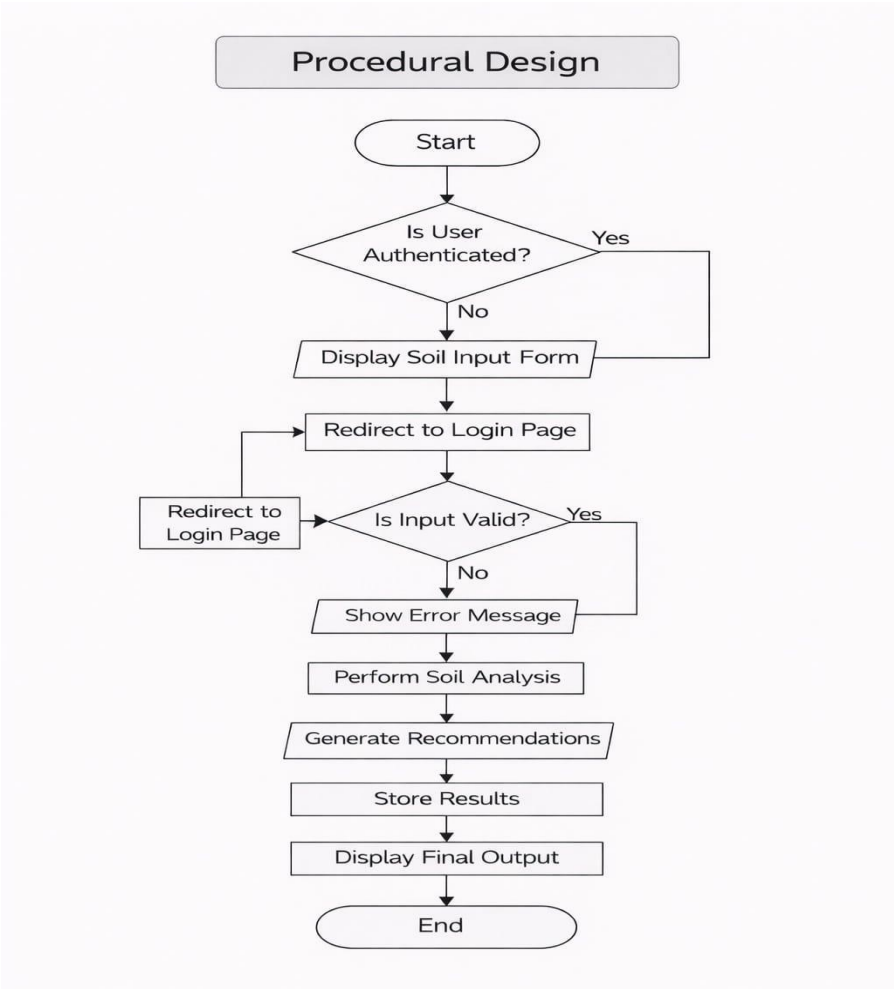
Non-Negative Constraint (Nutrient Values): Ensures that nutrient values like nitrogen, phosphorus, potassium, and moisture cannot be negative, as negative values are practically invalid.

Data Type Constraint: Enforces correct data types (e.g., INT, FLOAT, VARCHAR, DATE) to ensure proper storage and avoid data mismatch errors.

Referential Integrity Rule: Prevents deletion of a parent record (e.g., User) if related child records (Soil_Test) exist, unless properly handled, ensuring data consistency.

4.3.3 Procedural Design

Procedural design defines the step-by-step logic executed by the system.



vii 4.5 Procedural Design

Explanation: The procedural design control flowchart illustrates the step-by-step execution process of the Soil Testing Web Application. The process begins with system initiation and checks whether the user is authenticated. If the user is not authenticated, the system redirects them to the login page to ensure secure access. Once authentication is successful, the soil input form is displayed, allowing the user to enter parameters such as pH, nitrogen, phosphorus, potassium, and moisture. The system then validates the entered data to ensure all values are within acceptable ranges.

If the input is invalid, an error message is displayed and the user is prompted to correct the data. If the input is valid, the system proceeds to analyze the soil parameters, generate appropriate crop and fertilizer recommendations, and store the results in the database. Finally, the analyzed results are displayed to the user, and the process ends. This flowchart ensures a logical, structured, and secure execution of system operations.

4.3.4 Data Structures Used

o support efficient execution and structured data handling, the following data structures are used in the Soil Testing Web Application:

- User = {

```
    user_id: id,
    name: user_name,
    email: user_email,
    password: encrypted_password,
    registered_date: date
}
```

- SoilData = {
  pH: ph_value,
  nitrogen: n_value,
  phosphorus: p_value,
  potassium: k_value,
  moisture: moisture_value,
  test_date: date
  }

- AnalysisResult = {
  soil_condition: "Acidic / Neutral / Alkaline",
  nitrogen_status: "Low / Normal / High",
  phosphorus_status: "Low / Normal / High",
  potassium_status: "Low / Normal / High"
  }

- Recommendation = {
  crop_suggestion: crop_name,
  fertilizer_suggestion: fertilizer_type,
  additional_advice: improvement_suggestion
  }

- SoilTestRecord = {
  test_id: id,
  user_id: user_id,
  soil_data: SoilData,
  recommendation: Recommendation
  }

- API_Response = {
  status: "success / failure",
  message: system_message,
  data: {
    analysis: AnalysisResult,
    recommendation: Recommendation
  }
  }

This data structure is used for communication between frontend and backend via REST APIs. It ensures smooth data exchange in a structured and readable format.

4.3.5 Algorithm Design

Algorithm 1: Soil Parameter Evaluation and Crop Recommendation Algorithm

Input:
Soil parameters entered by the user (pH, Nitrogen, Phosphorus, Potassium, Moisture)

Output:
Suitable crop suggestion and fertilizer recommendation

Steps:

1. Start the algorithm.
2. Accept soil parameters from the user (pH, nitrogen, phosphorus, potassium, moisture).
3. Validate the input values.
4. If any value is invalid, display an error message and request correct input.
5. Determine soil condition based on pH value:
    - If pH < 6 → Soil is **Acidic**
    - If pH between 6 and 7.5 → Soil is **Neutral**
    - If pH > 7.5 → Soil is **Alkaline**
6. Evaluate nutrient levels:
    - If nitrogen < standard threshold → Mark as **Nitrogen Deficient**
    - If phosphorus < standard threshold → Mark as **Phosphorus Deficient**
    - If potassium < standard threshold → Mark as **Potassium Deficient**
7. Based on soil condition and nutrient status, select suitable crops from the recommendation database.
8. Generate fertilizer suggestions to improve soil quality.
9. Store the analysis result and recommendation in the database.
10. Display the recommended crop and fertilizer to the user.
11. End the algorithm.

Algorithm 2: Soil Test Record Management Algorithm

Input:
User login credentials and soil test data

Output:
Stored soil test record and displayed analysis results

Steps:

1. Start the algorithm.
2. Check whether the user is authenticated.

3. If the user is not authenticated, redirect to the login page.
4. If authentication is successful, allow access to the soil input form.
5. Accept soil parameter values from the user.
6. Validate all input fields to ensure values are within acceptable ranges.
7. Pass the validated data to the soil analysis module.
8. Generate crop and fertilizer recommendations using the analysis algorithm.
9. Create a soil test record containing:
    - User ID
    - Soil parameters
    - Generated recommendations
    - Test date
10. Store the record in the database.
11. Retrieve the stored results and display them on the user interface.
12. End the algorithm.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

The implementation of the Soil Testing Web Application was carried out using a modular and structured development approach. The system was divided into different modules such as user management, soil data input, soil analysis, recommendation generation, and database management. Each module was implemented separately and later integrated to form the complete system.

The project followed a step-by-step development process beginning with system design and database creation. After the design phase, the backend functionalities were developed using Python and Flask, while the frontend interface was built using HTML, CSS, and JavaScript. API communication was implemented to enable interaction between the frontend and backend components.

This approach ensured better code organization, easier debugging, and improved maintainability. By implementing the system in modules, developers were able to test each part independently before integrating it into the final application.

Plan of Implementation

- Requirement Breakdown
  - Identify the main objectives of the Soil Testing Web Application.
  - Analyze user needs such as soil data input, soil analysis, and recommendation generation.
  - Break down the system into smaller functional requirements for easier implementation.
- Module Identification
  - Divide the system into logical modules such as User Management, Soil Data Input, Soil Analysis, Recommendation, and Database Module.
  - Assign specific responsibilities to each module to maintain clear functionality.
  - Ensure modules can operate independently while interacting with other modules.
- Step-by-Step Development
  - Implement the backend logic and database structure first.
  - Develop the frontend interface for user interaction.
  - Connect frontend and backend using APIs.
  - Gradually integrate modules to form a complete working system.
- Integration Phase
  Once all individual modules were developed, they were integrated to form a complete working system. The frontend interface was connected to the backend APIs, and database operations were tested to ensure that user data and soil test records were stored correctly. Integration ensured that all modules functioned together seamlessly.

- Testing and Debugging
  After integration, the system was thoroughly tested to identify errors or inconsistencies. Debugging techniques were applied to fix issues related to input validation, API communication, and database operations. This phase ensured that the application functions correctly and provides accurate soil analysis results.

Standards Followed

- **Coding Standards**
  - Follow consistent naming conventions for variables, functions, and files.
  - Maintain proper indentation and formatting for readability.
  - Add comments to explain important sections of the code.
- **Modular Design Standards**
  - Design the system so that each module performs a single specific task.
  - Ensure modules can be modified or updated without affecting other parts of the system.
  - Promote code reusability and easier maintenance.
- **Iterative Development Standards**
  - Develop the system in multiple stages rather than all at once.
  - Test each module after development before integration.
  - Continuously improve the system based on testing results and feedback.

5.2 Overview of the Implementation

The implementation of the Soil Testing Web Application was carried out in a phased and systematic manner to ensure smooth development and proper integration of all system components. The development process was divided into multiple stages, where each stage focused on implementing specific functionalities of the system. This structured timeline helped the development team track progress, manage tasks efficiently, and ensure that all features were completed within the planned schedule.

Initially, the project began with the setup of the development environment and the creation of the basic application structure. After establishing the project framework, the focus shifted toward implementing the core logic for soil data processing and recommendation generation. As development progressed, additional modules such as database connectivity, data storage, and user interface integration were added to enhance the functionality of the application.

Throughout the implementation process, each module was developed and tested individually before being integrated into the complete system. This incremental development approach helped in identifying and resolving errors at an early stage. After implementing all modules, the final phase involved integrating the frontend, backend, and database components, followed by performance optimization and system testing to ensure that the application operates efficiently.

Implementation Timeline

Implementation Timeline

Weeks 1–2: Project Setup and Basic Structure

- Development environment was prepared and required software tools were installed.
- The basic structure of the application was created.
- Folder structure for frontend, backend, and database components was organized.
- Initial configuration of the Flask server was completed.
- Database connection setup was established.

Weeks 3–4: Soil Input and Evaluation Logic

- Forms were developed for users to enter soil parameters such as pH, nitrogen, phosphorus, potassium, and moisture.
- Input validation mechanisms were implemented to prevent incorrect values.
- Backend logic was created to process soil parameter inputs.
- Soil evaluation rules were implemented based on agricultural standards.

Weeks 5–6: Recommendation Module

- Crop and fertilizer recommendation system was implemented.
- Rule-based logic was designed to analyze soil nutrient levels.
- The system generated suggestions for suitable crops and fertilizers.
- Recommendation results were connected with the soil analysis module.

Week 7: Database Integration

- Database tables were created for users, soil test records, and recommendations.
- Backend functions were implemented for data storage and retrieval.
- Soil analysis results and user information were stored in the database.
- Secure data management and database connectivity were ensured.

Week 8: Final Integration and Optimization

- All system modules were integrated into the complete application.
- Communication between frontend and backend was finalized.
- System performance optimization was carried out.
- Bugs and errors were identified and fixed.
- Improvements were made to the user interface and error handling mechanisms.

5.3 Development Tools and Technology Stack

Tools and Technologies Used

The Soil Testing Web Application was developed using a combination of modern programming languages, frameworks, databases, and development tools. These technologies were carefully selected to ensure efficient system development, smooth performance, and easy maintenance of the application. By using widely adopted technologies, the system can be easily understood, modified, and extended in the future if new features need to be added.

The development of the system required both frontend and backend technologies to handle different aspects of the application. The frontend technologies were used to design the user interface and allow users to interact with the system, while backend technologies were responsible for processing soil data, performing analysis, generating recommendations, and managing communication with the database. This separation between frontend and backend helped in maintaining a clear system structure and improved the overall performance of the application.

In addition to programming languages and frameworks, several development tools and software environments were used to simplify coding, debugging, and project management. These tools provided features such as syntax highlighting, error detection, and version control, which helped developers write cleaner and more reliable code. Version control tools also ensured that different versions of the project could be tracked and managed efficiently.

Another important component of the technology stack is the database system, which is responsible for storing and managing application data. The database stores information such as user registration details, soil test parameters, analysis results, and crop or fertilizer recommendations. Using a structured database allows the system to retrieve and update data quickly while maintaining data consistency and security.

- **Python**
  Python was used as the primary backend programming language. It is widely used for web development and data processing due to its simple syntax, flexibility, and large ecosystem of libraries.
- **Flask Framework**
  Flask is a lightweight Python web framework used to build the backend of the application. It was used to create APIs, handle HTTP requests, process soil data, and manage communication between the frontend and the database.
- **HTML**
  HTML (HyperText Markup Language) was used to design the structure of the web pages. It defines the layout of forms, input fields, buttons, and result display sections.
- **CSS**
  CSS (Cascading Style Sheets) was used to style the web interface and improve the visual appearance of the application. It helps create a user-friendly and responsive design.
- **JavaScript**
  JavaScript was used to add interactivity to the user interface. It helps in handling form submissions, validating user inputs, and dynamically displaying results without refreshing the entire page.
- **SQLite / MySQL (Database)**
  A relational database system was used to store user details, soil test data, and recommendation results. The database ensures structured data storage and efficient retrieval of information.
  - **VS Code (Visual Studio Code)**
    Visual Studio Code was used as the main development environment for writing and managing the project code. It provides features such as syntax highlighting, debugging tools, and extensions for faster development.
  - **Git and GitHub**
    Git was used for version control to track changes in the code during development. GitHub was used for storing the project repository and collaborating on the project.

Justification for Tool Selection

The technologies used in this project were selected based on their simplicity, reliability, and compatibility with web-based application development.

Python was chosen because it is easy to learn, highly readable, and widely used in data processing and backend development. Its extensive libraries and community support make it suitable for building scalable applications. Flask was selected as the web framework because it is lightweight and flexible, allowing developers to build web applications quickly without unnecessary complexity.

HTML, CSS, and JavaScript were used for frontend development because they are standard technologies for building web interfaces. These technologies help create interactive and visually appealing user interfaces that can be accessed through any web browser.

The relational database system was used to ensure organized storage of application data such as user details, soil test records, and recommendations. Databases provide data consistency, security, and efficient retrieval of information when required.

Visual Studio Code was selected as the development environment because of its powerful features, easy integration with programming languages, and support for extensions that improve developer productivity. Git and GitHub were used to maintain version control and ensure that the project code remains organized and properly documented.

Overall, the selected technology stack provides a reliable and efficient platform for developing the Soil Testing Web Application while ensuring scalability, maintainability, and ease of future enhancements.

5.4 System Architecture

Architectural Approach

The Soil Testing Web Application follows a client–server architecture combined with a modular system design. In this architecture, the system is divided into two main parts: the frontend (client side) and the backend (server side). The frontend is responsible for providing the user interface through which users interact with the application, while the backend handles the processing of soil data, analysis logic, and communication with the database.

1. Presentation Layer (Frontend)

The frontend layer is developed using HTML, CSS, and JavaScript. This layer provides forms and interactive components that allow users to enter soil parameters such as pH level, nitrogen, phosphorus, potassium, and moisture. It also displays the analysis results and recommendations generated by the system. The frontend communicates with the backend through HTTP requests and APIs, enabling the system to send and receive data efficiently.

2. Application Layer (Backend)

The backend layer is implemented using Python and the Flask framework. This layer acts as the core processing unit of the system. It receives the soil parameter data from the frontend, performs soil analysis based on predefined rules, generates crop and

fertilizer recommendations, and interacts with the database to store and retrieve data. The backend ensures that the application logic is executed correctly and efficiently.

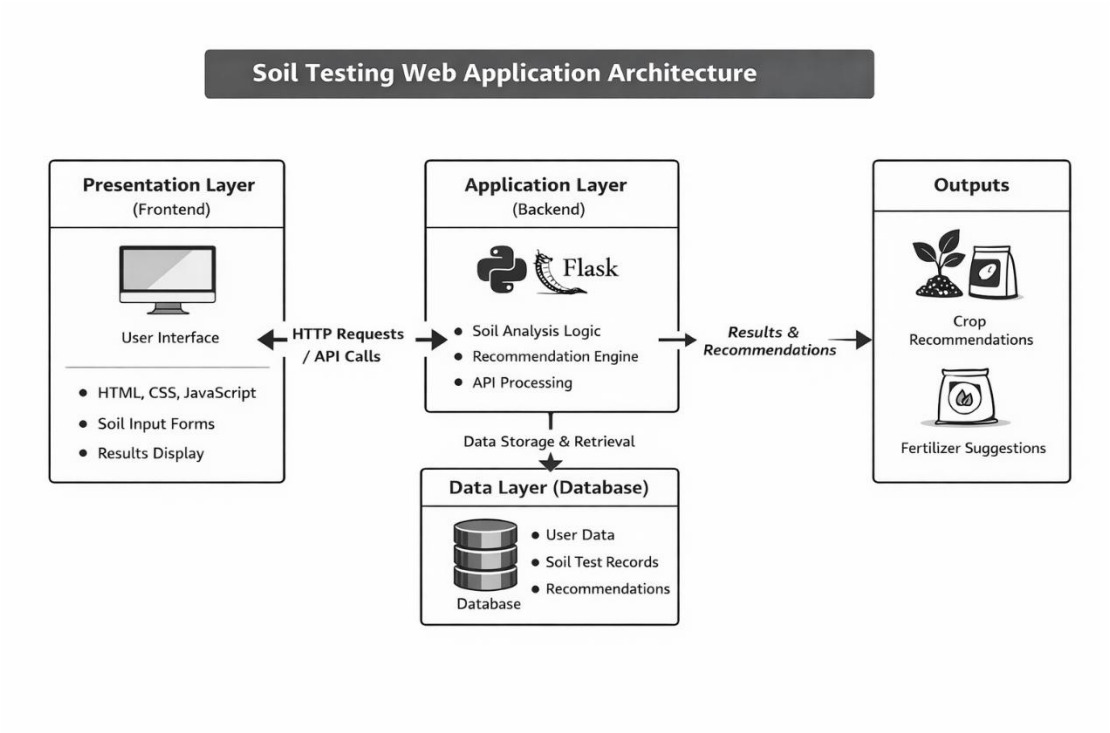
3. Data Layer (Database)

The database layer is responsible for storing all persistent data used by the system. It maintains records of registered users, soil test results, and recommendation outputs. The backend communicates with the database using database queries to insert, update, and retrieve information whenever required. This ensures that user data and soil test records are stored securely and can be accessed whenever needed.

Overall, this three-layer architecture—Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database) ensures clear separation of responsibilities within the system. Each layer performs a specific role, making the application more organized, scalable, and easier to maintain.

Benefits of the Chosen Architecture

- Scalability
  Since the frontend, backend, and database operate as independent components, new features or modules can be added without affecting the entire system. For example, future enhancements such as machine learning-based soil prediction or weather-based crop recommendations can be integrated easily.
- Improved Maintainability
  The modular design allows developers to modify or update specific parts of the system without making changes to the entire application. This simplifies system maintenance and future upgrades.
- Better Debugging and Testing
  If an issue occurs in one module, it can be identified and fixed without affecting other components of the system. This modular structure makes debugging and testing more efficient during development.
- Enhanced Data Security
  Sensitive information such as user credentials and soil test records is stored securely in the database. Data access is controlled through backend processes, reducing the risk of unauthorized access.
- Efficient Performance
  By separating responsibilities among different layers, the system can process requests more efficiently. The frontend handles user interaction while the backend processes logic and database operations.
- Future Expandability
  The architecture allows the system to be expanded easily in the future. Additional features, modules, or technologies can be integrated without major structural changes to the existing system.
- Separation of Concerns
  The client–server architecture separates the user interface, application logic, and database management into different layers. This separation makes the system easier to develop, update, and maintain.



viii 5.1 Soil Testing Wen Application Architecture

5.4 Module-wise Implementation

The Soil Testing Web Application is implemented using a modular development approach, where the entire system is divided into multiple functional modules. Each module is designed to perform a specific task within the application, which helps in improving the overall organization, maintainability, and scalability of the system. By separating the system into independent modules, developers can design, implement, and test each component individually before integrating them into the complete system.

The module-wise implementation ensures that different functionalities of the application such as user interface design, soil data processing, recommendation generation, and database management are handled in a structured manner. Each module communicates with other modules through defined interfaces, allowing smooth data flow between the frontend, backend, and database layers. This approach reduces system complexity and simplifies debugging and maintenance.

The major modules implemented in the system include User Interface Design, Functional Processing Modules, Database Integration, and Input/Output Handling. The user interface module manages the interaction between the user and the system, while the functional modules handle soil data analysis and recommendation generation. The database module stores user information and soil test records, and the input/output module manages data validation and result presentation.

Overall, the module-wise implementation provides a clear system structure, efficient development process, and easier testing, ensuring that each component of the Soil Testing Web Application functions correctly and integrates smoothly with the overall system.

5.4.1 User Interface Design

Layout and Screen Structure

- The user interface is designed with a simple and structured layout to make the system easy to use.
- The application contains screens such as Home Page, User Login/Register Page, Soil Data Input Form, and Results Page.
- The input screen allows users to enter soil parameters such as pH, nitrogen, phosphorus, potassium, and moisture.
- The results screen displays soil analysis results along with recommended crops and fertilizers.
- Navigation between pages is designed to be smooth so users can easily move between input and results sections.

Theme and Visual Design

- The application uses a clean and minimal visual design to improve readability and usability.
- A simple color scheme and structured form layout are used to ensure that users can easily understand the interface.
- Input fields, buttons, and result sections are clearly separated to avoid confusion.
- The interface is designed to be responsive, allowing it to work properly on different screen sizes and devices.

Tools Used

- **HTML** – Used to create the basic structure of the web pages.
- **CSS** – Used to design the layout, styling, and visual appearance of the interface.
- **JavaScript** – Used to add interactivity and handle client-side validation of input fields.
- **Flask Templates (Jinja2)** – Used to connect the frontend interface with backend logic.

5.4.2 Functional Modules

Soil Data Input Module

- Responsible for collecting soil parameters from the user through an input form.
- Allows users to enter soil values such as pH level, nitrogen, phosphorus, potassium, and moisture content.
- Ensures that all required fields are filled before submitting the data.
- Performs input validation to check that the values entered are within acceptable ranges.
- After validation, the data is sent to the backend system for further analysis and processing.

Soil Analysis Module

- Processes the soil parameters received from the input module.
- Compares the entered values with predefined agricultural standards.
- Determines the overall soil condition such as acidic, neutral, or alkaline based on pH values.
- Identifies nutrient levels and classifies them as low, normal, or high.
- Generates soil analysis results that will be used by the recommendation module.

Recommendation Generation Module

- Generates suitable crop and fertilizer recommendations based on soil analysis results.
- Uses rule-based logic to match soil conditions with appropriate crop types.
- Suggests fertilizers required to improve soil nutrient levels.
- Helps users understand how to enhance soil fertility and crop productivity.

Report Generation Module

- Displays the final soil analysis results and recommendations to the user.
- Organizes the output in a clear and structured format.
- Shows important details such as soil condition, nutrient levels, recommended crops, and fertilizer suggestions.
- Ensures that the results are easy for users to understand and interpret.

User Management Module

- Handles user registration and login authentication.
- Manages user sessions to ensure secure access to the system.
- Allows users to store and access their soil test records.
- Ensures that only authorized users can use the system and view their stored data.

5.4.3 Database Integration

Database Design

The system uses a relational database to store user information, soil test records, and recommendation results. The database ensures that all data is stored securely and can be retrieved whenever needed. The backend communicates with the database using queries to perform operations such as inserting new soil test data, retrieving previous reports, and updating user information.

Table Name	Field Name	Data Type	Description
User	user_id (PK)	INT	Unique identifier for each user
User	name	VARCHAR(100)	Name of the user
User	email	VARCHAR(100)	Unique email address
User	password	VARCHAR(100)	Encrypted password
Soil_Test	test_id (PK)	INT	Unique identifier for soil test
Soil_Test	user_id (FK)	INT	Links soil test to user
Soil_Test	pH	FLOAT	Soil acidity level
Soil_Test	nitrogen	FLOAT	Nitrogen level
Soil_Test	phosphorus	FLOAT	Phosphorus level
Soil_Test	potassium	FLOAT	Potassium level
Soil_Test	moisture	FLOAT	Soil moisture level
Soil_Test	test_date	DATE	Date of soil test
Recommendation	recommendation_id (PK)	INT	Unique ID for recommendation

Table Name	Field Name	Data Type	Description
Recommendation	test_id (FK)	INT	Links to soil test
Recommendation	crop_suggestion	VARCHAR(200)	Suggested crop
Recommendation	fertilizer_suggestion	VARCHAR(200)	Suggested fertilizer

d 5.1 Database Structure

Data Storage and Retrieval

- The system stores user information, soil test data, and recommendations in a structured relational database.
- When a user submits soil parameters, the backend processes the data and stores the results securely in the database.
- The application retrieves stored data whenever users need to view previous soil test reports or recommendations.
- Efficient database queries are used to insert, update, and retrieve records quickly and reliably.

5.4.4 Input/Output Handling

Input Handling

- The system collects soil parameters through input forms available on the web interface.
- Users enter values such as pH, nitrogen, phosphorus, potassium, and moisture levels.
- Client-side validation using JavaScript ensures that the entered values are numeric and within acceptable ranges.
- After validation, the data is sent to the backend server through API requests for processing and analysis.

Output Handling

- After the soil analysis process is completed, the system generates the results.
- The application displays soil condition information along with recommended crops and fertilizers.
- The results are presented clearly on the results page to ensure easy understanding for users.
- The generated output can also be stored in the database for future reference.

5.5 Challenges Faced and Solutions Implemented

During the development of the Soil Testing Web Application, several technical and design challenges were encountered. These challenges mainly involved handling user inputs, implementing soil analysis logic, ensuring smooth frontend–backend communication, and managing database connectivity. Since the system processes important soil parameters to generate recommendations, maintaining data accuracy and reliability was an important concern.

To address these issues, various solutions were implemented such as input validation, rule-based analysis logic, secure database integration, and proper API communication between system components. Security measures were also applied

to protect user data and restrict unauthorized access. By identifying these challenges early and implementing appropriate solutions, the overall performance, reliability, and usability of the system were significantly improved.

1. Invalid User Input

Challenge:
While entering soil parameters, users sometimes provided incorrect values such as negative nutrient levels or pH values outside the valid range. Since soil analysis depends on accurate input data, incorrect values could lead to wrong recommendations. Therefore, ensuring that the system accepts only valid and meaningful data became an important challenge during development.

Solutions:
- Implemented client-side validation using JavaScript to check input values.
- Added backend validation in Flask to verify the data again before processing.
- Restricted pH values between 0 and 14.
- Prevented negative values for soil nutrients.

2. Database Connectivity Issues

Challenge:
During the development phase, establishing stable communication between the backend application and the database was initially difficult. Connection errors or incorrect configurations sometimes caused problems while storing or retrieving data. Proper database connectivity was necessary to ensure that soil test records and user information were stored correctly.

Solutions:
- Configured proper database connection settings in the backend.
- Used reliable database libraries for handling connections.
- Implemented error handling mechanisms for database operations.
- Tested database queries to ensure correct data storage and retrieval.

3. Handling Missing or Incomplete Data

Challenge:
Another challenge occurred when users submitted forms without filling all the required fields. Missing parameters could lead to incomplete soil analysis and incorrect recommendations. Therefore, it was necessary to ensure that all required inputs were provided before processing the data.

Solutions:
- Added required field validation in the input forms.
- Displayed warning messages when fields were left empty.
- Prevented form submission until all required values were entered.
- Implemented backend checks to verify the completeness of data.

4. Implementing Soil Analysis Logic

Challenge:
Developing the logic for soil analysis required defining correct threshold values for nutrients and pH levels. Without proper reference standards, it would be difficult to

determine soil conditions and generate accurate recommendations for crops and fertilizers.

Solutions:
- Defined soil nutrient ranges based on agricultural standards.
- Implemented rule-based conditions for soil evaluation.
- Tested the analysis logic using sample soil data.
- Adjusted threshold values to improve recommendation accuracy.

5. Frontend and Backend Integration

Challenge:
Ensuring smooth communication between the frontend interface and the backend server was another challenge during implementation. Data entered by users needed to be correctly transmitted to the server, processed, and returned with results without errors.

Solutions:
- Created API endpoints using Flask for communication.
- Used HTTP requests to send data from the frontend to the backend.
- Implemented structured JSON responses for results.
- Tested API communication to ensure accurate data exchange.

6. User Authentication and Data Security

Challenge:
Protecting user information and ensuring that only authorized users can access the system was an important requirement. Without proper authentication and security measures, user data and soil test records could be exposed to unauthorized access.

Solutions:
- Implemented a secure login and registration system.
- Stored passwords in encrypted format.
- Added session management to maintain user authentication.
- Restricted system access to logged-in users only.

5.6 Testing Strategy and Methodology

Testing is an essential phase in the development of the Smart Soil Testing and Monitoring System. It ensures that the application functions correctly, processes soil data accurately, and maintains reliable communication between the frontend interface and backend services.

Since the system includes modules such as soil data entry, database storage, API communication, and result retrieval, a structured testing methodology was followed to validate the overall system.

The testing process involved Unit Testing, Integration Testing, System Testing, API Testing, Data Validation, and Error Handling checks. These testing methods ensured that each module performs as expected and that the complete application works smoothly without failures.

The main objective of testing was to verify data accuracy, proper database interaction, correct API responses, and a user-friendly interface.

5.6.1 Unit Testing

Unit testing was performed to verify the functionality of individual components of the application. Each module was tested independently to confirm that it processes input data correctly and produces the expected output.

Modules Tested

• Soil data submission function
• API endpoint for storing soil data
• Database insertion function
• Retrieval of soil test records
• Validation of soil parameters

Each component was tested using both valid and invalid input values to ensure reliability and proper error handling.

Example

Soil Data Submission Unit Test

Input
• Valid pH value
• Valid moisture percentage
• Correct nitrogen, phosphorus, and potassium values

Expected Result
• Soil test record successfully stored in the database

Result
• Passed

Invalid Input Test

Input
• Missing soil parameter values

Expected Result
• Error message displayed and data not stored

Result
• Passed

Unit testing helped identify issues such as:

• Incorrect parameter validation
• Missing input fields
• Data type mismatch errors
• Incorrect database query execution

5.6.2 Integration Testing

Integration testing was conducted to ensure that different modules of the application interact correctly with each other.

The Smart Soil Testing system includes multiple components such as the frontend user interface, Flask backend APIs, and the MySQL database. Integration testing confirmed that these components exchange data properly.

Modules Integrated

• React frontend forms with Flask backend APIs
• Flask backend with MySQL database
• Soil data input with database storage
• API response handling in the frontend

Example Integration Flow

1. User opens the soil testing interface
2. Soil parameters are entered in the form
3. Data is sent to the Flask API through an HTTP request
4. Backend processes and validates the data
5. Soil data is stored in the MySQL database
6. User retrieves stored soil test records

Integration testing helped detect issues such as:

• Incorrect API endpoint configuration
• Data format mismatch between frontend and backend
• Database connection errors
• Missing or delayed API responses

5.6.3 System Testing

System testing was performed on the entire application to ensure that all modules function together as a complete system.

The testing process included checking the full workflow from entering soil data to storing and retrieving results from the database.

Features Tested

• Soil data entry interface
• Soil parameter validation
• Storage of soil test results
• Retrieval of stored soil data
• API communication between frontend and backend
• Error handling and user feedback

The application was tested in a real browser environment to simulate normal user interactions.

Test Environment

Operating System
• Windows

Backend Framework
• Flask (Python)

Frontend
• React.js with Tailwind CSS

Database
• MySQL

Browser
• Google Chrome

All modules were tested to ensure correct functionality, performance, and stability of the Smart Soil Testing application.

5.6.4 Test Cases with Expected and Actual Results

To ensure the reliability and accuracy of the **Smart Soil Testing and Monitoring System**, all APIs were tested using tools such as Postman and browser-based requests. Both valid and invalid test scenarios were executed to verify system behavior, input validation, API responses, and database operations.

The testing process focused on confirming that the system correctly stores soil data, retrieves records, and handles user interactions without errors.

Testing covered:

• User Registration (valid and invalid inputs)
• User Login authentication
• Soil Test Data Submission (POST)
• Soil Test Data Retrieval (GET)
• Soil Statistics module
• Weather data retrieval
• Error handling and validation mechanisms

A. Registration API Testing (POST – Valid Input)

The registration module allows new users to create an account in the Smart Soil Testing system. Testing was performed by sending valid user details to the backend API.

API URL:
http://127.0.0.1:5000/register

Method: POST

Body Type: x-www-form-urlencoded

Parameters Sent

• name
• email
• password

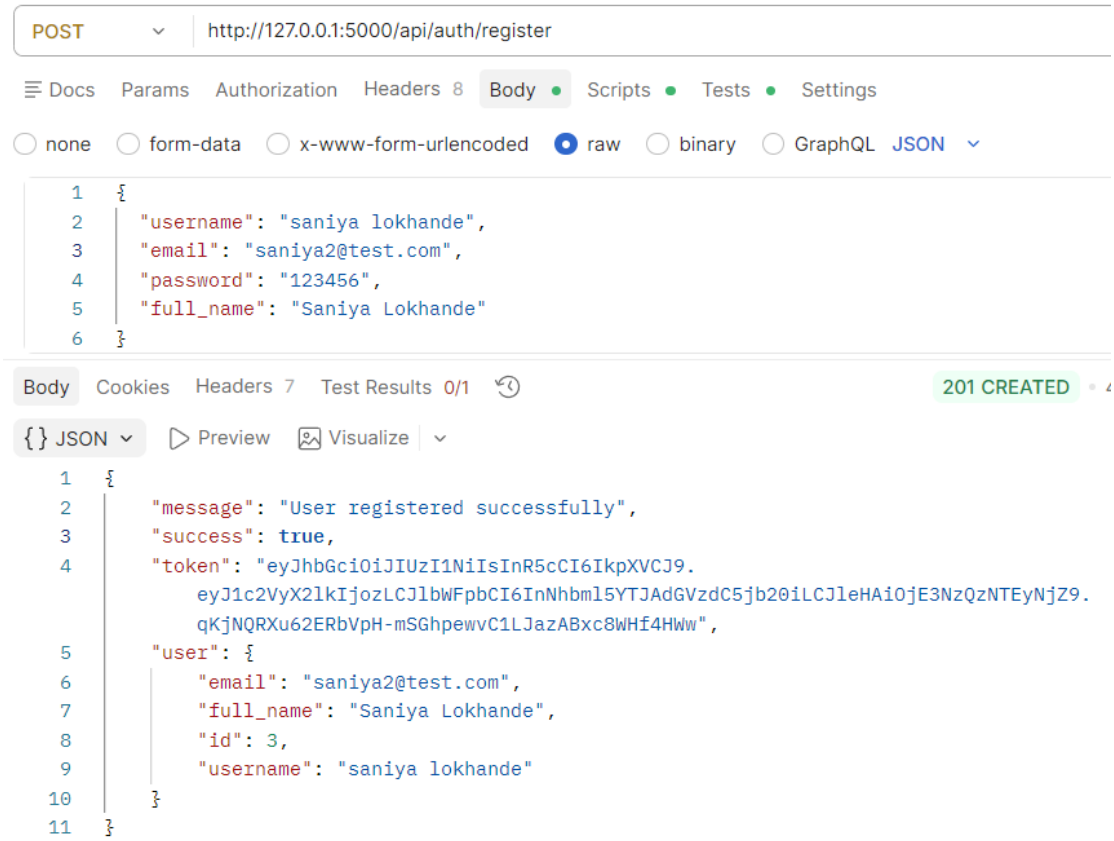
Expected Result

- User should be registered successfully
- Data should be stored in the database
- Status code 200 or 201 should be returned

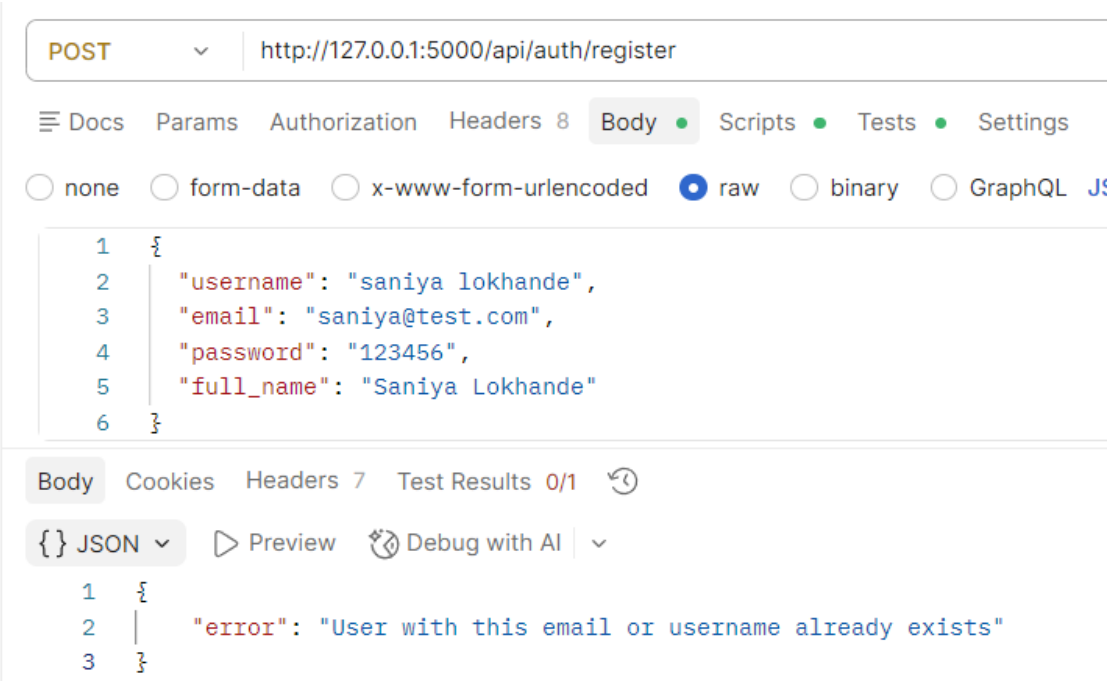
Actual Result

- User registered successfully
- Database entry created
- Status code 200 OK received

Status: Pass



i 5.1 Successful Registration



ii 5.2 Already Registered User

C. Login API Testing (POST)

The login module verifies whether registered users can access the system.

API URL:
http://127.0.0.1:5000/login

Method: POST

Parameters Sent

• email
• password

Expected Result

• User should be authenticated
• Access to dashboard should be granted
• Status code 200 returned

Actual Result

• Login successful
• Session/token generated
• Status code 200 OK received

Status: Pass

POST http://127.0.0.1:5000/api/auth/login

Docs Params Authorization Headers 8 Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON


```
{
  "email": "saniya@test.com",
  "password": "123456"
}
```


Body Cookies Headers 7 Test Results 1/1 200 OK

{} JSON Preview Visualize


```
{
    "message": "Login successful",
    "success": true,
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJ1c2VyX2lkIjoyLCJlbWFpbCI6InNhbml5YUB0ZXN0LmNvbSIsImV4cCI6MTc3NDM0ODEyOH0.
        qhmMwnRr9q-kD5FBDgWIt1ZgxDZPzqRncXcFycM5s6w",
    "user": {
        "email": "saniya@test.com",
        "full_name": "Saniya Lokhande",
        "id": 2,
        "location": "",
        "username": "saniya"
    }
}
```


iii *5.3 Login Successful*

D. Soil Test API Testing (POST)

This API is used to store soil test data entered by the user.

API URL:
http://127.0.0.1:5000/api/soil-test

Method: POST

Body Type: JSON

Parameters Sent

- pH
- moisture
- nitrogen
- phosphorus
- potassium
- temperature
- location

Expected Result

- Soil data should be stored in the database
- Status code 200 or 201 returned

Actual Result

• Soil test data stored successfully
• Database updated
• Status code received successfully

Status: Pass

POST http://127.0.0.1:5000/api/soil-test

Docs Params Authorization Headers 8 Body Scripts Tests

none form-data x-www-form-urlencoded raw binary

```
1  {
2    "ph": 6.5,
3    "moisture": 30,
4    "nitrogen": 40,
5    "phosphorus": 20,
6    "potassium": 35,
7    "location": "Mumbai"
8  }
```

Body Cookies Headers 7 Test Results 0/1

{} JSON Preview Visualize

```
1  {
2      "data": {
3          "location": "Mumbai",
4          "moisture": 30.0,
5          "nitrogen": 40.0,
6          "ph": 6.5,
7          "phosphorus": 20.0,
8          "potassium": 35.0,
9          "temperature": null,
10         "timestamp": "2026-03-17T16:53:45.017068"
11     },
12     "id": 3,
13     "message": "Soil test saved successfully",
14     "success": true
15 }
```

iv 5.4 Soil Test Entry

E. Soil Test Retrieval API (GET)

This API retrieves stored soil test data.

API URL:
http://127.0.0.1:5000/api/soil-tests

Method: GET

Expected Result

• Stored soil data should be returned
• JSON response should be received
• Status code 200

Actual Result

• Soil test records retrieved successfully
• JSON data displayed
• Status code 200 OK

Status: Pass

GET http://127.0.0.1:5000/api/soil-tests

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Body Cookies Headers 7 Test Results 1/1 200 OK

{} JSON Preview Visualize


```
1  {
2      "count": 3,
3      "data": [
4          {
5              "created_at": "2026-03-17T16:53:45",
6              "field_id": null,
7              "id": 3,
8              "location": "Mumbai",
9              "moisture": "30.00",
10             "nitrogen": "40.00",
11             "notes": null,
12             "ph": "6.5",
13             "phosphorus": "20.00",
14             "potassium": "35.00",
15             "temperature": null,
16             "test_type": "manual",
17             "timestamp": "2026-03-17T16:53:45",
18             "updated_at": "2026-03-17T16:53:45",
19             "user_id": null
20         },
21         {
22             "created_at": "2026-03-17T16:05:16",
23             "field_id": null,
24             "id": 2,
25             "location": "Mumbai",
26             "moisture": "30.00",
27             "nitrogen": "40.00",
28             "notes": null,
29             "ph": "6.5",
30             "phosphorus": "20.00",
```


ν 5.5 Soil Test Records

F. Statistics API Testing (GET)

The statistics module provides summarized or analyzed soil data for better understanding.

API URL:
http://127.0.0.1:5000/api/statistics

Method: GET

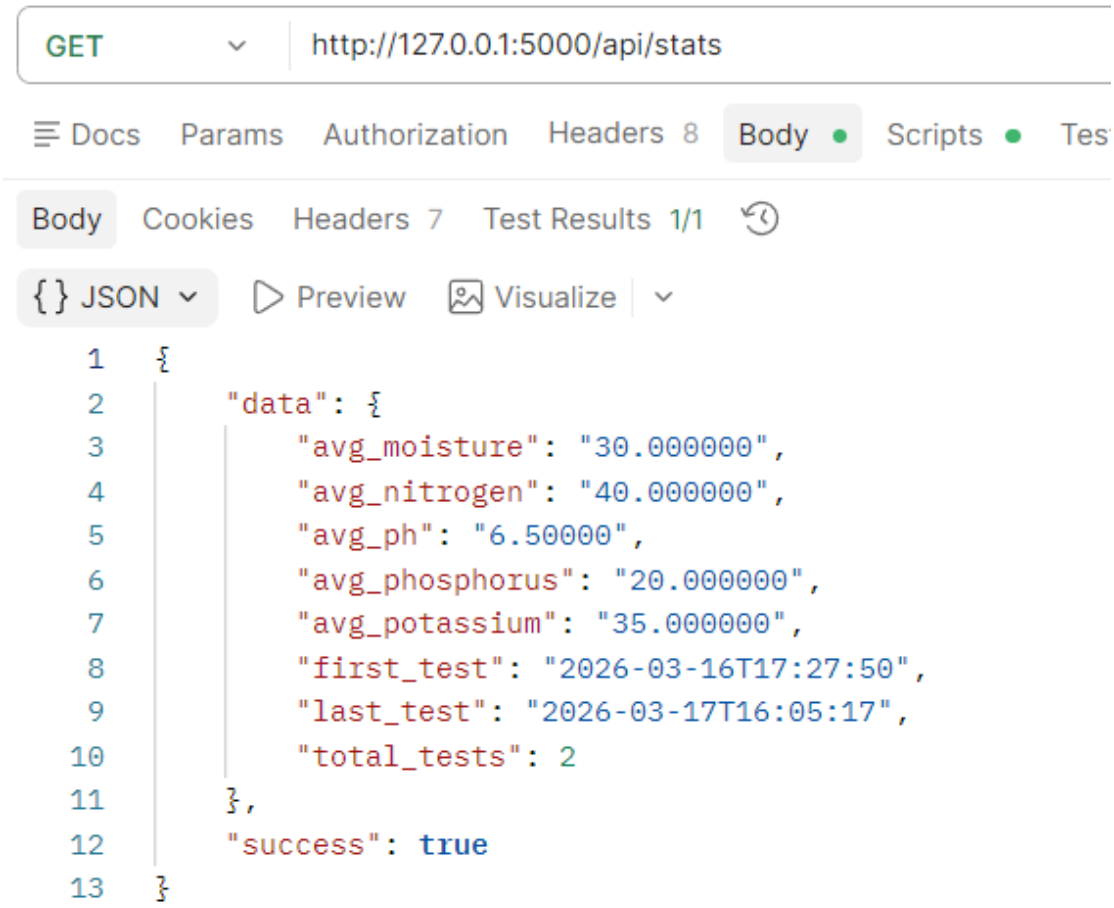
Expected Result

• Aggregated soil data should be returned
• Data should be displayed in charts/graphs
• Status code 200

Actual Result

• Statistical data retrieved successfully
• Graphs displayed correctly on frontend
• Status code 200 OK

Status: Pass



vi 5.6 Soil Statistics

G. Weather API Testing (GET)

The weather module provides environmental data that may affect soil conditions.

API URL:
http://127.0.0.1:5000/api/weather

Method: GET

Parameters Sent

• location (city or coordinates)

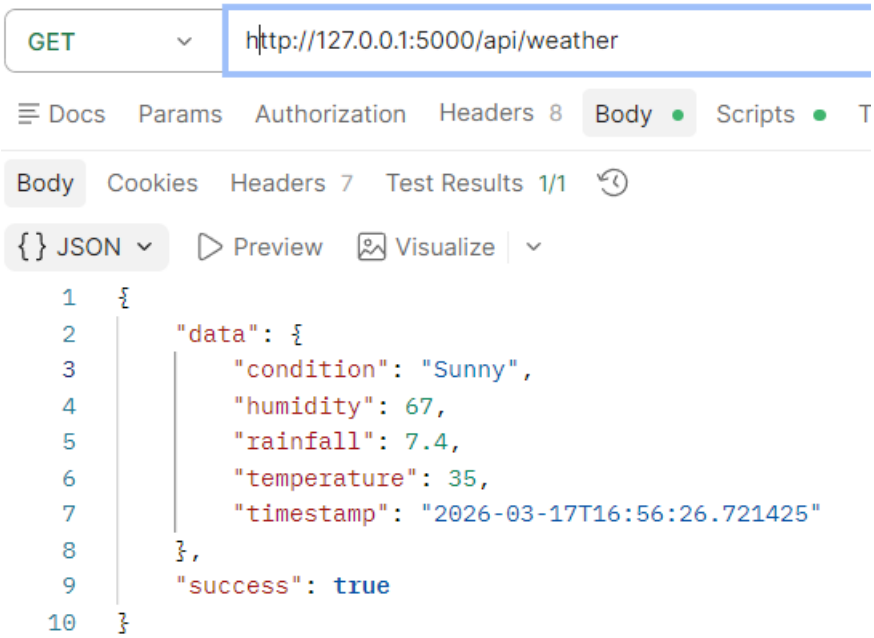
Expected Result

• Weather data should be fetched successfully
• Temperature and conditions displayed

Actual Result

• Weather data retrieved successfully
• Proper response displayed on UI
• Status code 200 OK

Status: Pass



vii 5.6 Weather Testing

G. Error Handling Test (Database Error Scenario)

During testing of the Smart Soil Testing system, a database-related error was encountered while performing multiple API operations simultaneously.

Error:

MySQL Operational Error / Connection Error

Cause

• Multiple API requests accessing the database at the same time
• Improper handling of database connections
• Connection not closed after execution

Fix Applied

• Implemented proper connection handling
• Closed database connections after each request
• Used connection pooling / retry mechanism
• Added error handling in Flask API

Result

• Error resolved successfully
• Database operations working smoothly
• No data loss observed

Status: Fixed

Summary of Test Results

Module	Result
Registration	Pass
Login	Pass
Soil Test POST	Pass
Soil Test GET	Pass
Statistics	Pass
Weather	Pass
Validation	Pass
Error Handling	Fixed

ℯ 5.2 Summary

All APIs of the Smart Soil Testing system were successfully tested using Postman and browser-based testing. The system correctly processes both valid and invalid inputs, ensures accurate storage and retrieval of soil data, and maintains database consistency across all operations.

CHAPTER 6: RESULTS

6.1 Test Reports

The Smart Soil Testing and Monitoring System was tested using both Postman and browser-based testing methods to verify the correctness, performance, and reliability of all modules. The testing process included validating API responses, checking database operations, and ensuring smooth interaction between frontend and backend components.

A structured testing approach was followed to evaluate the system under different conditions. Various types of testing such as functional testing, validation testing, API testing, and system testing were performed to ensure that each component behaves as expected. Special attention was given to data accuracy, response time, and error handling mechanisms.

All major functionalities such as user registration, login, soil data submission, data retrieval, statistics generation, and weather information were tested under different scenarios. Both valid and invalid inputs were used to evaluate system behavior and to ensure that the application can handle unexpected or incorrect data efficiently.

The system was also tested for data consistency and integrity, ensuring that soil test records are correctly stored and retrieved from the database without loss or duplication. Additionally, multiple API requests were executed to observe system stability under continuous usage.

Error handling mechanisms were thoroughly verified to ensure that meaningful error messages are displayed to the user without affecting system performance. The application was able to prevent invalid operations and guide users with proper validation messages.

The frontend interface was tested for usability and responsiveness to ensure a smooth user experience across different operations such as form submission, navigation, and data visualization. The integration between frontend and backend was confirmed to be seamless, with accurate data exchange through APIs.

The test results confirmed that the system performs efficiently, maintains data reliability, and handles errors properly without crashing. Overall, the Smart Soil Testing system proved to be stable, user-friendly, and capable of supporting real-time soil data management and analysis.

Module	**Test Case**	**Input/Action**	**Expected Output**	**Actual Output**	**Observation**
Auth	Valid Login	Enter valid data	Dashboard loads	Loaded successfully	Smooth login
Auth	Invalid Login	Wrong password	Error shown	Error displayed	Clear message
Auth	Empty Login	Empty fields	Warning shown	Warning displayed	Blocks invalid
Auth	Logout	Click logout	User logged out	Logout successful	Safe session
Soil Test	Add Data	Valid inputs	Data saved	Stored successfully	Accurate storage

Module	Test Case	Input/Action	Expected Output	Actual Output	Observation
Soil Test	Invalid Data	Missing values	Error shown	Error displayed	Proper validation
Soil Test	View Records	Open page	Records shown	Data displayed	Easy access
Soil Test	Delete Record	Delete entry	Record removed	Deleted successfully	Good control
Statistics	View Stats	Open stats page	Graphs shown	Charts displayed	Clear view
Statistics	Data Accuracy	Load data	Correct results	Accurate output	Reliable
Statistics	Refresh	Reload page	Data retained	Data retained	Stable data
Weather	Get Weather	Enter location	Weather shown	Data displayed	Real-time
Weather	Invalid Input	Wrong location	Error shown	Error handled	Strong handling
Weather	Refresh	Reload page	Updated data	Data refreshed	Dynamic update
System	DB Insert	Submit data	Data stored	Saved successfully	Reliable DB
System	Fetch Data	Get records	Data returned	Data retrieved	Consistent
System	Multi Request	Multiple API calls	No crash	Stable system	Good performance

ƒ 6.1 Test Report

6.1.1 Discussion of Test Results

- **User Interaction and System Flow**
  - **Smooth Navigation and Responsiveness:**
    All modules such as registration, login, soil test entry, statistics, and weather load efficiently with minimal delay. Page transitions are smooth and predictable, ensuring a seamless user experience without confusion or lag.
  - **User-Friendly Interaction:**
    The system provides simple forms and clear instructions, allowing users to enter soil data easily. Input fields are well-structured, reducing the chances of user errors during data entry.
  - **Edge Case Handling:**
    Scenarios such as page refresh, incomplete form submission, or navigation between modules are handled effectively. The system prevents data loss and ensures continuity of operations without crashes.

- **Data Validation and Error Handling**
  - **Input Validation:**

All user inputs such as soil parameters (pH, moisture, NPK values) are validated before submission. Incorrect or missing inputs trigger appropriate validation messages.

- **Error Handling Mechanism:**
  The system displays clear and informative error messages without exposing technical details. This ensures that users understand the issue and can correct it easily.
- **Prevention of Invalid Operations:**
  Invalid actions such as submitting empty forms or incorrect data formats are restricted, improving system reliability and data accuracy.

- **Soil Data Management System**
  - **Accurate Data Storage:**
    Soil test data entered by users is stored accurately in the MySQL database. Each record includes parameters such as pH, moisture, nutrients, and location.
  - **Reliable Data Retrieval:**
    Stored data can be retrieved quickly without delays. The system ensures consistency between stored and displayed data.
  - **Data Integrity:**
    No duplication or data loss was observed during testing. The database maintains consistency even during multiple operations.

- **Statistics and Analysis Module**
  - **Clear Data Visualization:**
    The statistics module displays soil data in graphical formats such as charts, making it easy for users to analyze trends.
  - **Accurate Calculations:**
    All statistical outputs are generated correctly based on stored data, ensuring reliable analysis.
  - **Performance Efficiency:**
    The module processes and displays data quickly, even with multiple records.

- **Weather Integration Module**
  - **Real-Time Data Fetching:**
    The system successfully retrieves weather information based on the user's location.
  - **Accurate Display:**
    Weather parameters such as temperature and conditions are displayed clearly on the interface.
  - **Error Handling:**
    Invalid locations or API issues are handled gracefully with appropriate messages.

- **Database Integration**
  - **Reliable Data Storage:**
    All system data, including user details and soil test records, are securely stored in the MySQL database.
  - **Efficient Query Handling:**
    Database queries are executed efficiently, ensuring fast response times.

- **Scalability:**
  The database structure supports future enhancements such as large-scale data storage, analytics, and integration with external systems.

- **UI/UX Performance and Accessibility**
  - **Responsive Design:**
    The interface adapts well across different screen sizes and devices, maintaining readability and usability.
  - **Consistent Layout:**
    Buttons, forms, and navigation elements follow a consistent design, reducing user confusion.
  - **Easy Navigation:**
    Users can easily switch between modules such as soil tests, statistics, and weather without difficulty.
  - **Interactive Feedback:**
    Immediate feedback is provided after actions such as form submission or errors, improving user experience.

- **Stability and Performance Testing**
  - **Handling Multiple Requests:**
    The system handled multiple API requests without crashes or performance issues.
  - **Continuous Usage Stability:**
    The application remained stable during prolonged usage, indicating efficient resource management.
  - **Robust Performance:**
    No system crashes, freezes, or unexpected errors were observed during testing.

Final Observation

Based on the detailed testing and analysis, the Smart Soil Testing and Monitoring System performs reliably and efficiently under various conditions. The system successfully handles valid and invalid inputs, maintains database consistency, and provides accurate results.

The results confirm that the application is stable, user-friendly, and suitable for real-world use in agricultural monitoring and soil analysis.

6.2 User Documentation

The Smart Soil Testing and Monitoring System is a web-based application designed to help users analyze and manage soil data efficiently. It provides a platform where users can input soil parameters, store data, view statistics, and access weather information.

The application is designed with simplicity and usability in mind, ensuring that users such as farmers, researchers, and agricultural professionals can easily interact with the system.

This documentation provides guidance on how to use the system effectively and understand its features.

6.2.2 System Requirements

6.2.2.1 Hardware Requirements

To ensure smooth operation, the system requires:

- Processor: Intel i3 or higher
- RAM: Minimum 4 GB
- Storage: 500 MB free space
- Display: 1024×768 resolution or higher
- Input Devices: Keyboard and mouse

6.2.2.2 Software Requirements

- Operating System: Windows / macOS / Linux
- Browser: Google Chrome, Firefox, Edge (latest versions)
- Backend: Python (Flask)
- Frontend: React.js with Tailwind CSS
- Database: MySQL
- Internet Connection: Required for weather API

6.2.3 System Overview

The system is divided into the following main modules:

- **User Authentication Module** – Handles registration and login functionality
- **Soil Test Module** – Allows users to enter and store soil parameters
- **Statistics Module** – Displays analyzed soil data in graphical form
- **Weather Module** – Provides real-time weather information
- **Database Module** – Stores and manages all application data
- **User Interface (UI)** – Provides a clean and responsive interface for user interaction

All modules work together to provide an efficient and user-friendly soil monitoring system.

6.2.4 User Authentication Module

The authentication module ensures secure access to the system by allowing only registered users to log in. It manages user registration and login processes while maintaining data security.

The module validates user credentials and prevents unauthorized access to the application.

6.2.4.1 Registration

The registration feature allows new users to create an account in the Smart Soil Testing system.

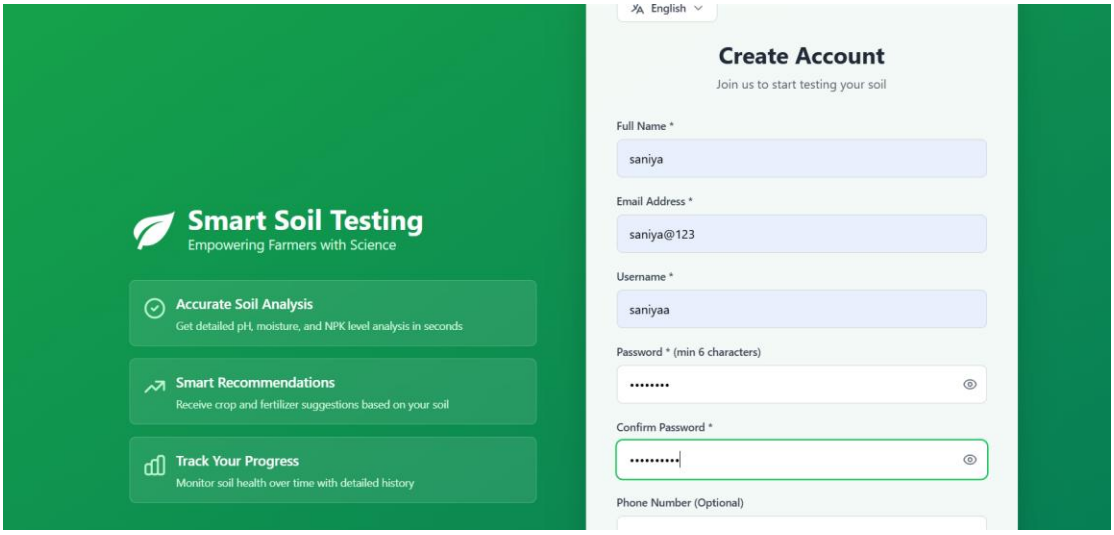
Steps to Register:

1. User opens the registration page
2. Enters required details such as name, email, and password
3. Submits the registration form
4. Data is sent to the backend API

5. Backend validates and stores the user information in the database
6. User receives confirmation of successful registration

Features:

- Prevents duplicate email registration
- Validates user input fields
- Displays error messages for invalid data
- Stores user data securely in the database



viii 6.1 Registration

6.2.4.2 Login

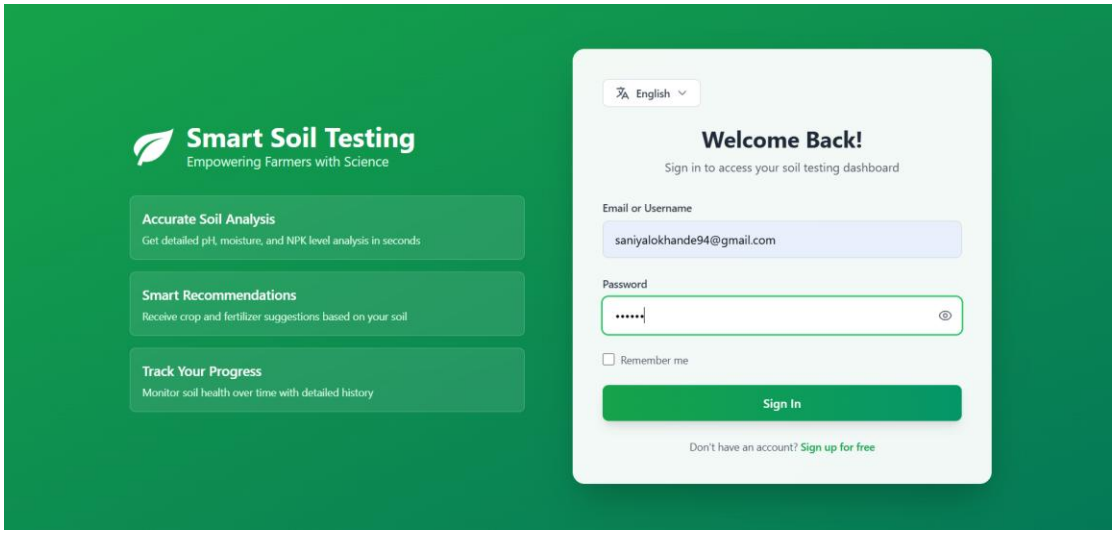
The login module allows registered users to securely access the Smart Soil Testing system.

Steps to Login:

1. User opens the login page
2. Enters registered email and password
3. Clicks on the login button
4. Credentials are sent to the backend API
5. Backend verifies the details from the database
6. If valid, user is redirected to the dashboard

Features:

- Secure authentication of users
- Error message for invalid credentials
- Prevents unauthorized access
- Session/token management for user access



ix 6.2 Login

6.2.5 Soil Test Module

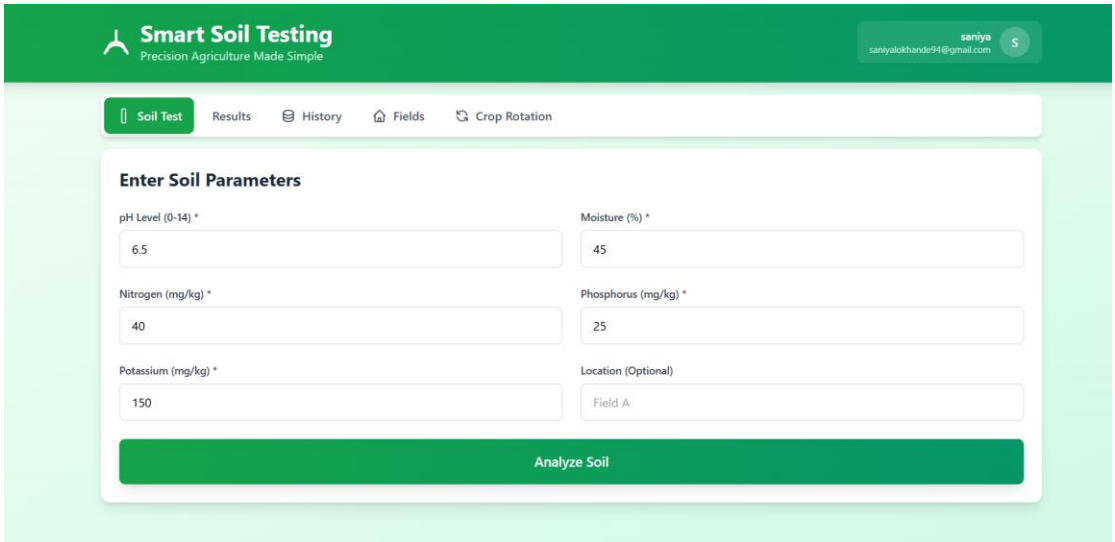
The Soil Test module is the core functionality of the system. It allows users to input and store soil parameters for analysis.

Steps to Use:

1. User navigates to the soil test section
2. Enters soil parameters such as:
   - pH
   - Moisture
   - Nitrogen
   - Phosphorus
   - Potassium
   - Temperature
3. Submits the data
4. Data is sent to the backend API
5. Backend stores the data in the database
6. User can view saved records

Features:

- Easy data entry interface
- Accurate storage of soil parameters
- Retrieval of previous soil test records
- Helps in monitoring soil health over time



x 6.3 Soil Test Module

6.2.6 Statistics Module

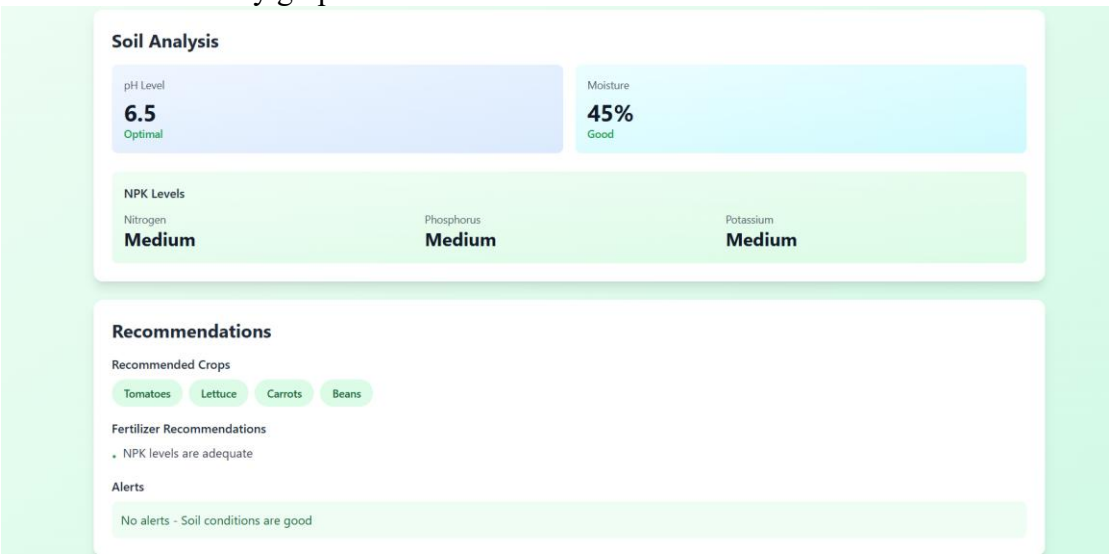
The Statistics module provides a summarized view of soil data, helping users analyze trends and patterns.

Steps to Use:

1. User navigates to the statistics section
2. System fetches soil data from the database
3. Data is processed and displayed in graphical format
4. User can analyze soil conditions easily

Features:

- Visual representation of soil data
- Helps identify trends in soil parameters
- Improves decision-making for agriculture
- User-friendly graphs and charts



xi 6.4 Statistics Module

6.2.7 History Module

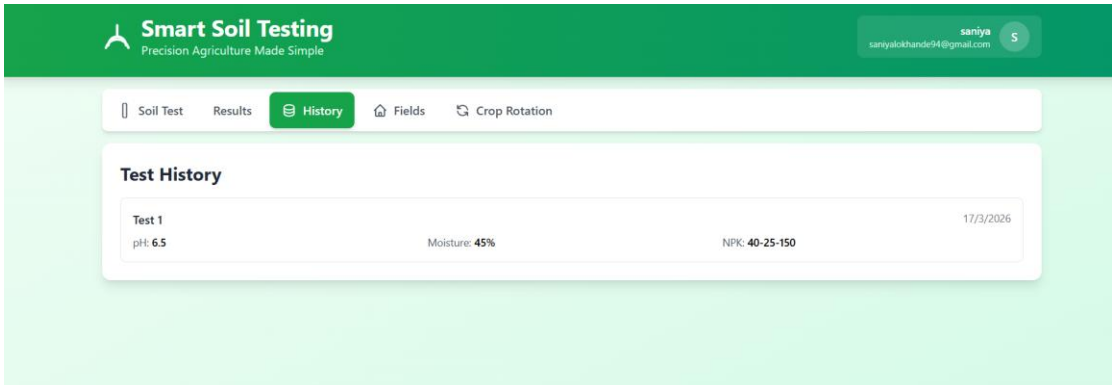
The History module allows users to view previously conducted soil tests and track past data for analysis and reference. This feature helps users monitor changes in soil conditions over time and make better agricultural decisions.

Steps to Use:

1. User navigates to the History tab
2. The system automatically retrieves stored soil test records
3. A list of previous tests is displayed
4. User can view details such as pH, moisture, NPK values, and test date

Features:

- Displays past soil test records in an organized format
- Shows important parameters like pH, moisture, and nutrient levels (NPK)
- Helps users track soil condition over time
- Data is fetched from the database instantly
- Simple and user-friendly interface



xii 6.5 History Module

CHAPTER 7: CONCLUSIONS

7.1 Significance of the System

The Smart Soil Testing and Monitoring System is a modern web-based application designed to assist users in analyzing and managing soil health efficiently. It integrates features such as soil data entry, data storage, statistical analysis, weather information, and user authentication, creating a complete platform for soil monitoring and agricultural decision-making.

During the development of the system, it was structured into multiple modules including User Authentication, Soil Test Management, Statistics, Weather Integration, and Database Management. These modules work together to provide a seamless and user-friendly experience for users such as farmers, researchers, and agricultural professionals.

Testing results demonstrated that the system is stable, responsive, and capable of handling various operations effectively. Users can easily enter soil parameters, store data, retrieve past records, and analyze soil conditions using graphical representations. The integration of weather data further enhances the usefulness of the system by providing environmental insights that impact soil health.

One of the key achievements of the system is its ability to digitize soil testing processes. Traditionally, soil data is recorded manually, which can lead to errors and inefficiencies. This system eliminates such issues by providing accurate data storage and quick retrieval, improving overall productivity.

From a technical perspective, the system successfully integrates a Flask backend with a MySQL database, ensuring reliable data storage and efficient API communication. The React-based frontend provides an intuitive and responsive interface, allowing users to interact with the system easily across different devices.

Overall, the Smart Soil Testing system achieves its objectives by providing a structured, efficient, and accessible platform for soil data management. It effectively combines technology and agriculture to support better decision-making and improved crop management.

7.2 Limitations of the System

Despite the successful implementation of the system, certain limitations were identified:

- Limited Data Analysis Features:
  - The system provides basic statistical visualization but lacks advanced analytics such as predictive modeling or crop recommendations.
  - More intelligent insights based on soil data are not currently implemented.
- Dependency on Manual Input:
  - Soil data must be entered manually by the user.
  - The system is not yet integrated with real-time IoT soil sensors for automatic data collection.
- Database Constraints:
  - The system uses a local MySQL database, which may limit scalability for large-scale deployments.
  - Multi-user synchronization and cloud-based access are not fully implemented.

- Weather API Dependency:
  - Weather information depends on external APIs.
  - In case of API failure or network issues, weather data may not be available.

- User Interface Limitations:
  - Some UI components may not be fully optimized for very small screens or low-resolution devices.
  - Additional accessibility improvements can be made for better usability.
- Testing Limitations:
  - Testing was conducted in a controlled environment with limited datasets.
  - Real-world testing with large-scale users and diverse conditions was not performed.
- Time Constraints:
  - Due to project limitations, advanced features such as AI-based recommendations and mobile app development were not implemented.

7.3 Future Scope of the Project

The Smart Soil Testing system has strong potential for future enhancements and expansion.

- AI-Based Recommendations:
  - Future versions can include AI models to suggest suitable crops and fertilizers based on soil data.
  - Predictive analysis can help users make better agricultural decisions.
- IoT Integration:
  - Integration with soil sensors can enable real-time data collection.
  - This will eliminate manual input and improve accuracy.
- Cloud-Based System:
  - Migrating to cloud storage will allow access from multiple devices.
  - It will also support large-scale data storage and multi-user environments.
- Mobile Application Development:
  - Developing a mobile app will allow users to access the system anytime and anywhere.
  - Notifications can be added for updates and reminders.
- Advanced Data Visualization:
  - More interactive graphs and dashboards can be implemented for better analysis.
  - Comparative and historical data analysis can be added.
- Integration with Weather Forecasting:
  - Future improvements can include detailed weather forecasts and alerts.
  - This will help users plan agricultural activities more effectively.
- Multilingual Support:
  - Adding multiple language options will make the system accessible to a wider audience, especially farmers from different regions.
- Large-Scale Testing and Deployment:
  - Conducting real-world testing with farmers and agricultural experts will improve system reliability and usability.

7.4 Practical Implications

The Smart Soil Testing system has several real-world applications:

- For Farmers:
  Provides an easy way to monitor soil health and make informed decisions regarding crop selection and fertilizer usage.
- For Agricultural Experts:
  Offers a digital platform to analyze soil data and provide recommendations based on accurate records.

- For Researchers:
  Helps in collecting and analyzing soil data for research and development purposes.
- For Government and Organizations:
  Can be used to monitor soil conditions on a larger scale and support agricultural planning.

By combining data management, analysis, and environmental insights, the system provides a practical solution for improving agricultural productivity.

In conclusion, the Smart Soil Testing and Monitoring System successfully fulfills its objective of providing an efficient and user-friendly platform for soil data analysis and management. The system is stable, reliable, and capable of handling real-world scenarios effectively.

Although certain limitations exist, they provide clear opportunities for future improvements. With the integration of advanced technologies such as AI, IoT, and cloud computing, the system has the potential to evolve into a powerful tool for modern agriculture.

Overall, this project demonstrates how technology can be effectively utilized to improve agricultural practices and support sustainable farming.

CHAPTER 8: REFERENCES / BIBLIOGRAPHY

Smart Soil Testing and Monitoring System

1. Food and Agriculture Organization (2025). *Soil Health and Sustainable Agriculture*. Retrieved from https://www.fao.org
2. Indian Council of Agricultural Research (2025). *Soil Testing and Crop Productivity Guidelines*. Retrieved from https://icar.org.in
3. National Informatics Centre (2025). *Soil Health Card Scheme*. Retrieved from https://soilhealth.dac.gov.in
4. World Bank (2024). *Agriculture and Soil Management Practices*. Retrieved from https://www.worldbank.org
5. United States Department of Agriculture (2025). *Soil Quality and Nutrient Management*. Retrieved from https://www.usda.gov
6. GeeksforGeeks (2025). *Flask Tutorial – Python Web Development*. Retrieved from https://www.geeksforgeeks.org/flask-tutorial/
7. React.js (2025). *Official Documentation*. Retrieved from https://react.dev
8. Flask (2025). *Flask Documentation*. Retrieved from https://flask.palletsprojects.com
9. MySQL (2025). *MySQL Documentation*. Retrieved from https://dev.mysql.com/doc/
10. Postman (2025). *API Testing Guide*. Retrieved from https://www.postman.com
11. OpenWeather (2025). *Weather API Documentation*. Retrieved from https://openweathermap.org/api
12. Ministry of Agriculture and Farmers Welfare (2025). *Soil Health and Agricultural Practices*. Retrieved from https://agricoop.nic.in
13. Brady, N. C., & Weil, R. R. (2016). *The Nature and Properties of Soils*. Pearson Education.
14. National Aeronautics and Space Administration (2025). *Soil Moisture and Climate Data*. Retrieved from https://www.nasa.gov
15. W3Schools (2025). *HTML, CSS, and JavaScript Tutorials*. Retrieved from https://www.w3schools.com

CHAPTER 9 : ANNEXURE

List of Tables

a. 4.1 User Table 55
b 4.2 Soil Test 56
c 4.3 Recommendation Table 56
d 5.1 Database Structure 71
e 5.2 Summary 84
f 6.1 Test Report 86

List of Diagrams

i. Gantt Chart 39
ii.PERT CHART 40
iii. 4.1 Use Case Diagram 46
iv. 4.2 Class Diagram 48
v. 4.3 ER Diagram 49
vi. 4.4 System Flowchart 51
vii 4.5 Procedural Design 58
viii 5.1 Soil Testing Wen Application Architecture 68

List of Screenshots

i 5.1 Successful Registration 77
ii 5.2 Already Registered User 78
iii 5.3 Login Successful 79
iv 5.4 Soil Test Entry 80
v 5.5 Soil Test Records 81
vi 5.6 Soil Statistics 82
vii 5.6 Weather Testing 83
viii 6.1 Registration 90
ix 6.2 Login 91
x 6.3 Soil Test Module 92
xi 6.4 Statistics Module 92
xii 6.5 History Module 93